International Designation:	BSH General Bus Specification D-Bus-2.2
National Designation:	BSH General Bus Specification D-Bus-2.2
Second International Designation:	BSH General Bus Specification D-Bus-2.2
Description:	
Owner:	Thorsten Piehler, PED-SDD3
Document Number:	55600000003459
Rev, Seq:	H3
CR-No.:	
CR-Rev:	
Document Type:	Requirement Specification
File Name:	55600000003459.docx

	Name	Department	Date
1. Checker			
2. Checker			
Responsible			
Released			
+Created			

Title in English: **BSH General Bus Specification D-Bus-2.2**

Title in German: BSH General Bus Specification D-Bus-2.2
Additional title: D-Bus-2.2 and Diagnostic Interface II

Description: *Requirements of a universal Bus System for connecting several micro controllers (UART based). The bus is multimaster capable. No duplex.*

Version: V1.22
Author: Thorsten Piehler, Przemyslaw Psonka, Erik Boritsch, Bernd Augustin, Tomas Hrasok, Samuel Baron
Contact point: PED-support@bshg.com
Date of creation: 20.12.2004
Last amendment: Samuel Baron
Amendment date: 21.06.2024

Eco: 2A00QG
Index: H3
Document number: 5560 0000003459
Number of pages: 86
Name: PED_General_Word

Informed	Name	Department	Contact
PCG	Konrad Zenz	GED-SDC2	+49 (8669) 30-2250 Konrad.Zenz@bshg.com
PDC	Woelfle Johannes	REU/OP-DCDES	+49 (9071) 521645 Johannes.Woelfle@bshg.com
PRF	Michael Huber	REU/OP-RFDGEA	+49 (7322) 923673 Michael.HuberMic1@bshg.com
PLC	Holger Herker	GED-SDC7	+49 (30) 81402-2800 Holger.Herker@BSHG.COM
PCP	Johannes Ewender	PCP-TDTES	+49 (8669) 30-5251 Johannes.Ewender@bshg.com
Software Architecture	Konrad Götz	GED-SEA	+49 (941) 8994-2066 Konrad.Goetz@bshg.com
Software Architecture	Klaus Dörfler	REU/OP-RFDGEA	+49 (7322) 92-3619 Klaus.DoerflerK@bshg.com
ePact Internal Communication	Klaus Dörfler	GED-SDD	+49 (7322) 92-3619 Klaus.DoerflerK@bshg.com
Creator & Released by	Thorsten Piehler	GED-SDD3	+49 (941) 8994-2168 Thorsten.Piehler@bshg.com

Abstract:
This document describes the requirements of a universal Bus System for connecting several electronic control units (UART based). The bus is multi-master capable. The bus is made for addressed communication. The bus is half-duplex capable.

Revision History:

Version	Change	Name	Date
1.0	English translation of the corresponding German document	Holen	30.10.02
1.1	Change after review	Holen	25.03.03
2.0	D-Bus-2 / Diagnostic Interface II Changes and extensions according to the new concept of D-Bus-2 Changes in front of and after review	Holen Holen	31.03.03 – 29.04.03 May- June.03
2.1	Minor corrections to maximum DataLength of Read/WriteRequest. Added main hardware node as configurable issue to Section 6.2 Application specific degree of freedom. Reference Errors repaired (Cross-references) Feedback from product divisions reviewed and adapted	Holen Holen	07.07.03 21.07.03
2.2	Adding chapter 3 Hardware	Götz	28.07.03
2.3	Formal work on Table Of Contents	Götz	28.07.03
2.4	Font size in figure 1 – Structure of D-Bus-2 – adapted. Table: reference to uchar IDBlock deleted from the list of parameters for Get ID Address. Added explicit description to SetMemoryModule: Module will not be reset by communication time-out, only after new SetMemoryModule, or a controller reset. Added Module to Identity Response Added revised HW chapter	Holen	25.08.03 07.10.03 08.10.03 09.10.03
2.5	Changed the description of section 5.2.3.3 GoOffline: After a node has entered off-line state, this node will never react to any bus messages again (until a reset has been executed).	Holen	23.10.03
2.6	Adaptions made to HW chapter Adapted the Version on the title page and the footer to a numerical field (can be edited in Word under document properties), in order to link the version on the title page with the version on the footer.	Zei Holen	29.10.03 13.11.03
2.7	-Added Copyrigths (page 2) -Removed length error as statistical error value (length error is redundant, as a too short frame would correspond to an acknowledgement time-out (or CRC error), and a too long frame would correspond to a CRC error. -Description of Msg Identity Response: Changed Parameter Module from Prog/SubSys to Module.	Holen	28.11.03
2.8	D-Bus II renamed to D-Bus-2 Section 5.2.2.4 Msg Identity Response: Length corrected to 6 Byte Section 5.5: Example: Communication with pc (Service Messages) Corrected the addition of Source (0xC0) to the identity request, and Module to the identity response. Section 5.2.1: Added a block of service messages, which may be used differently by different product divisions / products. ChangeRequest No. 2: Section 1.3 adapted (document number added) Chapter 3: New Hardware chapter (adaptations made regarding maximum communication partners possible)	Holen Holen Zei	20.01.04 21.01.04 21.06.04
2.9	Section: 7.6.1.1 and 7.6.1.2: Modified NodeGuardConfirm from one message per node (identifier being dependent on the node address) to one message for all slaves, including the node address as an index in the second data byte. This simplifies the implementation (saves ROM code).	Holen	25.06.04
2.10	Section 2.4 Idle recognition: Random waiting time before detection of idle state adapted after investigation of parameter influence.	Holen	04.10.04

2.11	Section 5.2.6: Added description of a Ping message for remote control. This allows e.g. SI to check whether the device is switched on (in which case the ping is acknowledged on data link layer level).	Holen	08.11.04
2.12	Incorporation of change requests as described in document: "EDS Change Proposals D-Bus-2.pdf" Added section 7.3.1 Chapter 6: Adapted description to: Returning of negative acknowledgement when input buffer is full.	Holen	30.03.05
2.13	Adaptations to Subsystem addressing after meeting with product divisions 20.04.05. Acknowledgement generation may be differentiated depending on the addressed subsystem. Please refer to section 2.3. Changed examples in Table 8, which can be misleading, as the examples previously used would fit better for describing use of MemoryModule. Section 6.1.1: Added a hint, that by default messages are prioritised according to the order of the definition (Subsystem 0 first).	Holen	25.04.05
2.14	Amendment: The following service messages are added as optional: 5.2.3.11 Msg EraseBlockRequest and 5.2.3.12 Msg EraseBlockResponse 5.2.3.10 Msg BlockSizeResponse has been amended with one additional optional byte, which tells the size of the block to be erased (for devices/memory modules which require blocks to be erased before writing to them). Amendments after D-Bus-2 meeting with product divisions (15.06.05) Amendment to chapter 2.4: Added to the specification that the collision counter is unaffected by negative acknowledgements (acknowledgement busy and acknowledgement wrong). Adaptation on front page (Erwin Fakesch is taking over from Günther Hammerl). Corrected PDM reference to Remote control extension (5560 instead of 5600) Removed reference 1 from Table 2 as this is obsolete. Updated references to this table. Corrected message length of Msg . Amended reference 2 (coexisting documents), which has recently been introduced to PDM. Chapter 5.2.3.14: Added a databyte: Sender, similar to the use of other request messages. Chapter 4.8: Amendment: Clarification of acknowledgement for communication partners, which responds to more than one address. Added following messages after EDS meeting December 2005: 5.2.3.15 Msg UpdateStatusRequest, 5.2.3.16 Msg UpdateStatusResponse and 5.2.3.17 Msg UpdateStatusReset. Figure 19 corrected (node 3 had a mirrored interface). From Section 3.8: Removed sentence: "Each node has at least 2 connectors to realise the D-Bus line structure." Chapter 5.4 adapted according to RemoteControlExtension. Chaper 5.2.2.4 adapted. Both 16 and 32 bit addressing can be used. Table updated. Chapter 3.8 adapted sentence about D-Bus-2 connections (minimum 2 for normal nodes). Minor adaptations after review in D-Bus-2 & EDS-Tools Working Group Added Chapter 9.	Holen Holen Holen Holen Holen Holen Holen Zei Holen Holen Holen Holen Holen	03.05.05 17.06.05 19.07.05 30.08.05 15.09.05 27.09.05 27.09.05 09.01.06 19.01.06 29.08.06 12.09.06 20.09.06 04.10.06 11.10.06

2.15	Hardware implementations (e.g. circuit diagrams) removed from specification. A new document will be prepared for different hardware implementations/recommendations: • hardware manual for specific hardware impementations and application notes. Contradictions removed. Rework of chapter 2 Minor adaptations of other chapters: • Chapter 4: Example and figure added. • Chapter 5: After Sales Services Communication objects and test messages added. • Chapter 7.6.2: Added Sleep mode messages. • Optional byte added to GoOffline message	 Bosen/Chomic Holen Holen Holen Holen	19.12.11 03.11.11 19.01.12 06.02.12 22.03.12 24.04.12
2.16	• Chapter 2.3 : changes related to node addressing: usage of nodes with same node address and different subsystems in order to increase the number of available nodes. • Chapter 5: ChangeProtocol message added • Chapter 5: GoOnline message added. • Changed document format to docx	Boritsch	27.01.17
Change to D-Bus 2.1			
1.0	• Change D-Bus Version to 2.1 • D_Bus 2.1 supports D-Bus 3 on the same line	Bernd Augustin (PED-DEAS)	28.04.15
1.1	• Edited Coexisting documents table • Added HSI to glossar • Added comment to the $t_{byte,max}$ in Chapter 4. • Fixed missing parts from 2.15.	Boritsch (PED-DEAK)	27.01.17
Change to D-Bus 2.2			
0.1	• Change D-Bus Version to 2.2 • First draft of documentation with commands for update • Precise shape of MAP commands for future remains to be clarified	Psonka	09.10.2017
0.2	• Ping Message F080	Psonka	20.10.2017
0.3	• More detailed description of Dbus2 Update and the process	Psonka	27.11.2017
0.4	• Review of V0.3	Psonka	28.11.2017
1.0	• First official version of the Dbus2.2 Spec	Psonka	1.12.2017
1.1	• From GoOffline, removed optional parameters, that never have been used or implemented anyway • Add more comments for IdentityRequest, SetMemoryModule, ReadResponse, ReadResponse32	Psonka Boritsch	6.12.2017 22.12.2017
1.2	• Added HSI Protocol Request and Response to Dbus2 Update Service Messages • Removed Service Messages Change Protocol and GoOnline from Spec, as not used or implemented anyway • Removed old Ping • More specific description of reset to leave offline mode	Psonka	22.02.2018
1.3	• Added “Return from Silent Mode” Request and Response • Added ECU Config Request and Response • More detailed description of possibility to perform parts of update with standard baud rate, in case precautions (provoking framing errors) should not work	Psonka Psonka	25.04.2018 27.04.2018
1.4	• Added 4ms Wakeup break signal to application layer (to be processed by electronic circuit, not software) • Added Wakeup Sent Request and Response to Optional Sercive Messages • Added to Wakeup Sent Response parameter Sender Address, to identify the one, who sent the wake up	Psonka Psonka	23.05.2018 25.05.2018

1.5	• Removed contradictions concerning max Dbus communication speed • Unified naming of Dbus to Dbus2/Dbus 2.2. Removed Dbus2.1 • Changed naming in the way, that the general term D-Bus-2 is used, where there is no difference with versions 2.1 and 2.2	Psonka	08.06.2018 11.06.2018
1.6	• More precise infos, what node addresses in responses mean (e.g. Identity Response) • Info, that service messages must be send by only one master at a time • More detailed description of handling of 4ms Wakeup Break • Make Wakeup Sent Request compulsory for every node, that received 4ms break. • Corrected numbering of drawings	Psonka	01.08.2018
1.7	• Changed length of WakeUp break to 15ms • Added parameter sender to ECU Config Read Response • Declared messages deprecated: BlockSizeRequest, BlockSizeResponse, EraseBlockRequest, EraseBlockRespoonse, StartStreamMode, EndStreamMode, UpdateStatusRequest, UpdateStatusResponse, UpdateStatusReset	Psonka	03.01.2019
1.8	• Marked NMT and Remote Control Messages as deprecated • Added Touch Messages, Common Components Messages and Functional Safety Messages • Extended list of coexisting Documents	Psonka Psonka	11.01.2019 14.01.2019
1.9	• In chapter 1, added info, that only certain baud rates are allowed and supported by implementation • Added references to Functional Safety Specification, chapter 1.3 • More precise description of generation of time for message repetitions, in chapter 2.4 • To WakeUpSentRequest, chapter 5.2.3, added more precise info about sender addresses and sending of the request • Added info about response timeouts to UpdateServiceMessages, where it has been missing, apart from “HsiProtocolRequest”, as this is dependent of embedded protocols, chapter 5.2.11 • Added hint to study effects on Functional Safety in case of Silent Mode because of interrupts, chapter 5.2.11 • Added more precise information about contents of “ECU Config Read Response”, chapter 5.2.11 • Corrected references to chapter of Update Service Messages from “5.2.8” to new number “5.2.11”. (This has been wrong in V1.8 of the Dbus2.2 specification)	Psonka	12.04.2019
1.10	• Added status “Update Mode Active” to “Ecu Config Read Response”. • Added reference to “Dbus Hardware Datasheet” and reduced chapter 3 (“Physical Layer”) to just reference that document.	Psonka	16.05.2019
1.11	• Added System Master Messages 0xA000 - 0xA1FF • Added comment confirming, that procedure used by implemented MAP master to put nodes into Update Mode is according to specification. • Added “Ecu Unique Id Read Request” and “Ecu Unique Id Read Response” to Optional Service Messages	Psonka	04.09.2019
1.12	• Added parameter “Sender Address” to “Ecu Unique Id Read Response”	Psonka	16.09.2019

1.13	• Option to increase waiting time after collision by a predefined constant for non service messages. • Added reference to official TeamCenter touch message specification. • Added SMM Device Calibration and Test Messages. • For “Ecu Unique Id Read Request” added info, that the Unique Id is in fact a Tracing Id. • Added remark, not to use Service Messages, for which a response is expected as broadcasts (unless explicitly stated otherwise for given message). • Defined weaker requirements for the usage of the WakeUp handshake. • Power Fail message	Psonka	08.01.2020
1.14	• Added MsgId range: Microcontroller Debug Messages • Added Information about BCD coding in Unique Id Read Response • Added Information about “Power Resurge Message” • Added Info, that Baud Rate Timer should stop during processing of “HSI Protocol Request” • Production Messages added. • Added info about Reset Break.	Psonka	11.05.2020
1.15	• More detailed description of params in Production Service Messages • Added MsgId range: SMM Test Server Touch Messages • Added MsgId range: Drives Control Diagnostic Messages • Added Production Messages for Appliance Data	Psonka	16.11.2020
1.16	• Formal corrections • Correction of start message Id of SMM Test Server Touch Messages • Update Info for “Unique Id Read Response”: No maximum length and explanation, what to do with different sorts of coding • Updated flow charts for “Update Service Messages”	Psonka	16.12.2020
1.17	• “Unique Id Read Response”: Remove limitation for string length to 30 • More precise info, concerning writing of Repaircount in production • More elaborating info about content of “Ecu Config Read Response” • Added MsgId range “Common Bluetooth Module Messages.”	Psonka	11.02.2021
1.18	• Explicit description, that HSI/MAP/FAP messages are only available in Update Mode • In application specific degrees of freedom, added recommended number of sending attempts. • Updated device types for Appliance Messages • Added Msg ID Range for “Generic Appliance Communication Messages” • Added new messages ”Ecu Software Submodule Read Request/Response”	Psonka	23.07.2021
1.19	• Added “BLE TX Power” element into ApplianceData	Hrasok	04.02.2022
1.20	• Added DBusCAN power management messages	Hrasok	04.10.2022
1.21	• Added DBus Factory reset message • Modified list of mandatory DBus messages • Added Msg ID Ranges for Sigma Data & Smart Sensor Bus	Hrasok	15.08.2023
1.22	• Modified description for Silent Mode for ECU(s) with DBusCAN chip	Baron	20.06.2024

Table 1: Revision History

Table of Contents

1 Introduction

This document describes the requirements of a universal bus system for diagnostic and internal communication purposes between several ECUs in BSH appliances. The universal bus system is called D-Bus-2, which contains Diagnostic Interface II as subset. The bus system is originating from the D-Bus-2 specification including specific items for diagnostic purposes.

D-Bus-2 features:

- Multimaster capable
- UART based
- One wire (communication)
- Supported communication speeds. The communication speed must be determined for each system, **allowed** and supported (see "Baud Rate Verify Request" and "Baud Rate Transition Request" in chapter 5.2.13) speeds are (in baud):
  - 9.6 k
  - 19.2 k
  - 38.4 k
  - 57.6 k
  - 115.2 k
  - 125 k
  - 230.4 k
  - 250 k
  - 500 k[1]
  - 1 M[1]
- For any project it is recommended that the planned communication speed can be configured to a higher standardised speed, to ensure that a need for a higher baud rate does not force a redesign of the system – beyond reconfiguration of the UART. In particular, when basing the development on 9600 baud, it is recommended that adjusting the speed to 19200 baud should be possible only by reconfiguring the UART.
- Scalability
  - Application layer can be added on demand.
  - Network management can be added on demand, the D-Bus-2 can be changed against e.g. CAN bus (without affecting the software using the bus).
- Message oriented
  - Target directed
  - CSMA/CD
  - 16 Bit Identifier, including a range for remote control extension (Internet@ppliances), test messages, service messages and normal process data exchange.
- Safety
  - 16 Bit CRC (Xmodem 16bit) – this breaks with the compatibility towards D-Bus-1.
  - The bus is laid out for addressed communication (half-duplex capable) – broadcasts are also possible
- Coexistence with D-Bus-3 on the same communication line

Source code (e.g. in C language) used in this document serves as an example of implementation, to illustrate the issues, and is not mandatory.

Since the original release of D-Bus-2, certain aspects have changed. Adding compatibility with D-Bus-3 has upgraded the version of D-Bus-2 to D-Bus-2.1. Adding functionalities for updates has caused an upgrade to D-Bus-2.2. Concerning the topics, where there is no difference between the versions, the following text will use D-Bus-2 for all versions D-Bus-2, D-Bus-2.1 and D-Bus-2.2.

1.1 Aim

An important issue of this specification is to make the diagnostic interface bus capable.
The upgrade from D-Bus-1 to D-Bus-2 makes it possible to implement the D-Bus in all product divisions. The bus serves as internal communication interface between ECUs as well as a standardised external interface (e.g. for

[1] Baudrates 500k and 1M available only for DBusCAN chip. However, DBusCAN chip does not support all baudrates listed. See DBusCAN chip specification in chapter 1.3 for more details.

diagnostic usage) for after sales services. Furthermore though the definition of the D-Bus for the next generation (D-Bus-3) it is mandatory to enhance the D-Bus-2 in a way that the D-Bus-2.1/D-Bus-2.2 and D-Bus-3 can be operated on the same communication line.

1.2 Limitations

D-Bus-2 is no longer compatible with D-Bus-1, due to implementation of the 16 bit CRC, which offers a higher safety level. See section 4.7 for further description of the CRC. D-Bus-2.1, D-Bus-2.2 and D-Bus-3 are compatible to each other. A D-Bus-2 node on a communication line makes the whole communication line incompatible to D-Bus-3.0 and can completely disrupt the communication.

1.3 Coexisting Documents

Reference	Description	Document name/ No.	Language	Review status	Responsible
1.	Remote Control Extension of D-Bus-2	D-Bus-2_RemoteControl.pdf 5560 0000003740	English	Released	Boritsch
2.	Electronic Identification of Software and hardware components in electronic control boards	ID-String.pdf 5560 0000004455	English	Released	Boritsch
3.	After Sales Service Communication Objects	DBus2AfterSalesServ.pdf 5560 0000007091	English	In work	Boritsch
4.	Hardware manual	D-Bus-2_HardwarManual.pdf 5560 0000010006	English	In work	Bosen, Chomic
5.	DBUS Hardware Datasheet	60100004727569_01.docx-A1-Q.pdf	English	In work	Sheraz Ahmed
6.	Functional Safety Message Server	FsMessageServer_60100002442818_01.docx-A1-Q.pdf	English	In work	Matthias Mursch
7.	Power Management Node Messages	60100000810931_Spec_PMN.docxs-B1-Q.pdf	English	In work	Simon Hallinger
8.	Touch Messages	60100005525969 DBus2 Touch Messages	English	In work	Peter Winter, Oliver Fischer
9.	Software Class B definition	Guideline: Funtional Safety by Software Class B 6010 0005038035	English	In work	Stefan Smetanka
10.	Functional Safety	Generic Functional Safety Specification 5560 0000007751	English	In work	Stefan Smetanka
11.	Functional Safety software	Functional Safety SW Design 5560 0000003770	English	In work	Stefan Smetanka
12.	Common Components Messages	https://production.github.bshg.com/pages/CommonComponents/docu/	English	In work	Przemyslaw Psonka
13.	SMM Device Calibration and Test Messages	https://production.polarion.bshg.com/polarion/#/project/SMCaTS/wiki/30%20-	English	In work	Dmytro Kravtsov

		%20Requirements/SMCaTS%20Software%20Specification			
14.	SMM Test Server Touch Messages	https://wiki.bsh-sdd.com/x/x5BOWw	English	In Work	Urwashi Kapasiya
15.	Drives Control Diagnostic Messages	https://production.polarion.bshg.com/polarion/#/project/ePactDrivesControl/workitem?id=ePDC-7650	English	In Work	Daniel Magura
16.	Generic Appliance Communication Messages	https://production.polarion.bshg.com/polarion/#/project/Architecture_Goals_Features_Abstracts/wiki/40%20-%20Architecture%20_%20Design/SDS_DBus2_Communication_Specification	English	In Work	Ralf Hochhausen
17.	Test specification	Deprecated. Has never been officially released in a place accessible to everybody.	English	In work	Ulrike Brüggemann
18.	D-Bus-3 specifications	Deprecated. Has never been officially released in a place accessible to everybody.	English	In work	Bernd Augustin
19.	DBusCAN System Basic Chip	https://wiki.bsh-sdd.com/display/ASSA/DBusCAN+System+Basic+Chip	English	In work	Kai Kropp
20.	Sigma Data	https://production.polarion.bshg.com/polarion/#/project/Cooling_Software_Library/wiki/Collectors/Domain_Sigma_Data	English	In work	Tabak Bohus

Table 2: Coexisting Documents

1.4 Terms and Abbreviations

Bus	Common line used for communication between different electronic partners.
Communication partner	Node on the bus.
ECU	Electronic Control Unit
Node	Electronic unit connected to the bus for communication purpose
Main node	Node, which provides power to the bus (V_{cc_bus}) as well as keeps D-line on 5 V during idle state (via pull-up resistor).
Sub node	All nodes, which are not main nodes.
UART	Universal Asynchronous Receiver/Transmitter
CSMA/CD	Carrier Sense Multiple Access / Collision Detection: A set of rules determining how network devices respond when two devices attempt to use a data channel simultaneously (called a collision). Standard Ethernet networks use CSMA/CD. This standard enables devices to detect a collision. After detecting a collision, a device waits a random delay time and then attempts to re-transmit the message. If the device detects a collision again, it waits twice as long to try to re-transmit the message. This is known as exponential back off.
Full Duplex	The possibility to Receive and Send simultaneously.
Half-duplex	The possibility to either Receive or Send (during reception it is not possible to send)
Ongoing Receive	Some controllers (e.g. Freescale HCS08) has got a flag (Ongoing Receive), which

	is logical TRUE as long as a reception is in progress. NB! This is not the inverted information of the idle flag offered by most controllers!
Asynchronous Serial Interface	Data is transmitted one byte at a time by asynchronous serial transmission. There is no determined timeslot between each byte, therefore a Start Bit and a Stop Bit is needed to signalise the receiver that the transmission starts or finishes.
Idle	Momentarily not active. D-Bus-2 has static 5 V level on D-line when idle.
D-Line	Communication line for D-Bus-2 (data line).
Data Collision	Two (or more) independent signals interfering on one physical line. This can be in a network, when two nodes start sending simultaneously after idle.
Bit-time	The time needed to write one bit. At 9600 Bit/s: 1/9600 s.
Baud rate	Communication speed.
Bit rate	Communication speed. For D-Bus-2 the bit rate corresponds to the baud rate (one symbol per bit).
Frame	Data transmission block
Message	Data content of a frame
ACK/ Acknowledgement	Low-level confirmation (in this specification in general an acknowledgement refers to OSI/ISO Reference Layer 2 acknowledgement)
MemoryModule	Software arrangement within a communication partner. This may refer to an explicit memory module (e.g. internal EEPROM) – but it can also be a logical part of a larger memory area (i.e. part of another MemoryModule).
SubSystem	An addressable part within a communication partner (for D-Bus-1 this was called module).
SW	Software
HW	Hardware
CRC	Cyclic redundancy check. Used for error detection/correction.
Polynomial	Arithmetical series of expressions, e.g. $a + bx + cx^2$.
Broadcast	No specific target addressed. All communication partners receive the message. There is no acknowledgement from the receivers of a broadcast (the sender generates the acknowledgement).
Big-Endian	Used for describing in which order data is saved/transmitted. Big-Endian derives from "Big end in first". This means that in the case of a two byte integer the high byte will be transmitted first, and the lowbyte as second byte (high value on lower address; low value on higher address).
CAN	Controller Area Network. A bus developed by BOSCH. Extensive use in the automotive Industry, but also in other industries the CAN bus gains popularity due to its low price based on the high mass production.
NMT	Network Management..
Node	Electronic unit connected to the bus for communication purpose. Synonym to Communication Partner. In NMT-perspective a node is a slave.
Remote Node	The network management master keeps an image of each remote node (slave) on the network.
DLL	Data Link Layer. This is the software layer closest to the hardware (or the Hardware Abstraction layer), according to the OSI/ISO reference model.
Nibble	4 bits of a byte.
MSB	Most significant bit
LSB	Least significant bit
RF	Radio frequency
HSI	High Speed Interface used for flashing with higher bandwith
DBusCAN	An ASIC chip, that provides interface between MCU and DBus/CAN comm. line

2 D-Bus-2 Structure

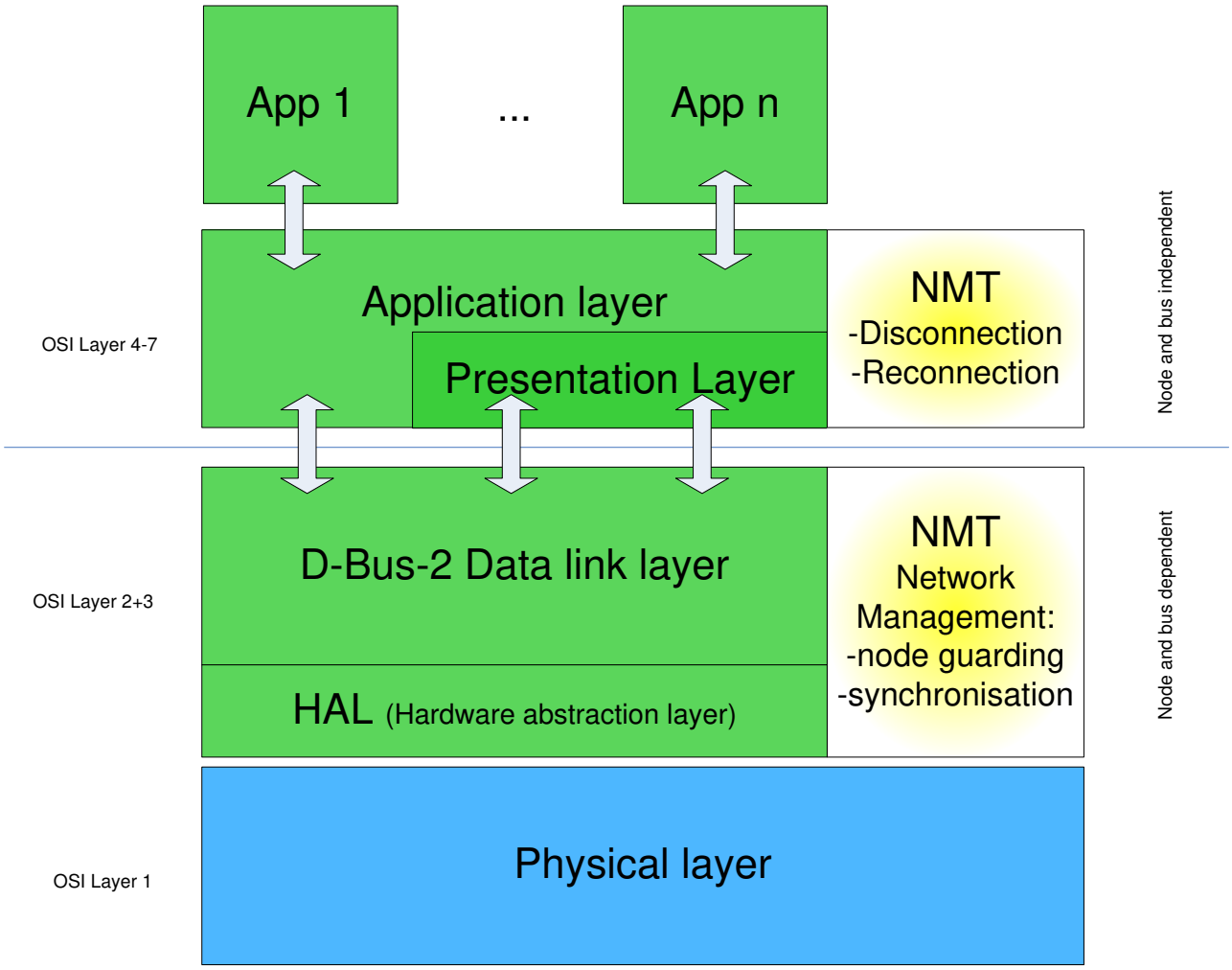


Figure 1: D-Bus-2 Structure

2.1 Levels of implementation

2.1.1 Minimum Requirement – OSI Layers 1 and 2

D-Bus-2 can be reduced to comprehend the OSI layers 1 and 2 in addition to the presentation layer for the diagnostic interface (see chapter 5) and possibly a remote extension. In this case the application must have knowledge about the bus structure[2]. In this case the bus cannot easily be exchanged (e.g. towards a CAN bus).

2.1.2 Application Layer

If the application layer is fully implemented, D-Bus-2 can be exchanged with any other bus (assuming that the other bus has got the same interface towards the application – characterised by vertical arrows in the figure above) without the need of adapting the application[3].

[2] The D-Bus-1 (predecessor of this bus) covers only OSI layer 1 and 2, hence the application needs knowledge about the bus system.
[3] The application layer from one bus system will be exchanged with the one from the other bus system.

2.1.3 Maximum Requirement – Application Layer and Network Management

If needed, network management can be added. This part (fade-yellow filled blocks in Figure 1 above) is optional. Network Management may be restricted to lower level implementation. If, however, the application layer implementation is chosen, the usage of the bus (including network management) is independent of the chosen structure of the bus. Unlike the minimum requirement, including only OSI Layers 1 and 2, this "full" implementation allows an implementation of the application independent of the underlying bus – the application does not depend on information whether the bus is a D-Bus-2 or CAN bus. See chapter 7 for a description of NMT, including different levels of NMT-implementation.

2.2 Data Transmission

Each communication partner may transmit data on the bus. The sender must wait for idle detection of the bus (except before generating an acknowledgement).

After detecting idle state on the bus, the sender transmits all the data of one frame continuously. In the receiver part the sender detects whether the correct data is actually sent on the bus. If the data is incompatible with the transmitted data, a data collision is detected. The transmission is halted, and repeated after idle detection.

The condition for idle detection is marginally delayed by each communication partner (each communication partner has a different delay) to avoid potential deadlocks.

If transmission of a frame was successful, the sender gets an acknowledgement from the receiving communication partner. If the acknowledgement is missing, the frame is marked as not sent. The timing is described in section 4.2.

2.3 Data Reception

Each communication partner (participant on the bus) always listens to the traffic on the bus. If a frame is addressed to this communication partner, the frame is saved and acknowledged. Is the frame meant for another communication partner, it is discarded (and not acknowledged).

It is possible to have multiple nodes with the same node address, which respond to different subsystems only. A whitelist of subsystems should be specified for each such node. In this scenario an acknowledge is only generated for a listed subsystem in order to prevent multiple ACKs.

If a time-out occurs during reception of a frame ($t > t_{ibyte,max}$), the frame is discarded.
The receiving node generates an acknowledgement either unconditionally (according to scenario 2 in **Figure 4**) or depending on the addressed subsystem[4]. A node may differentiate between subsystems before generating the acknowledgement, similar to scenario 1. It is possible to generate the acknowledgement for a given subsystem (and broadcast) only. Another node (scenario 2) may accept only subsystem 0 (e.g. a sensor node, which in the future is expected to exist as a slightly different variant, responding to subsystem 1 – same communication partner address).

[4] Whether the acknowledgement is generated unconditionally for a correctly addressed node (independent of the subsystem) depends on the system architecture. In general this aspect is configurable.

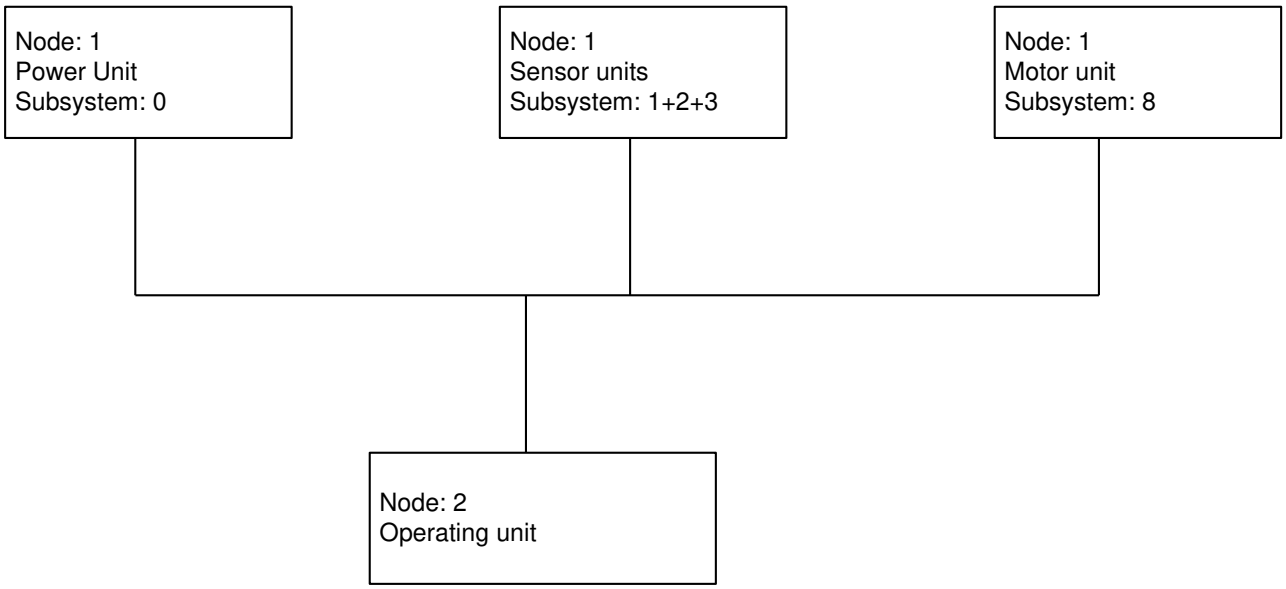


Figure 2: Subsystem differentiation scenario 1.

A third variant may generate acknowledgement to all frames addressed to the current node, independent of the subsystem used. This constitutes the default behaviour of a node.

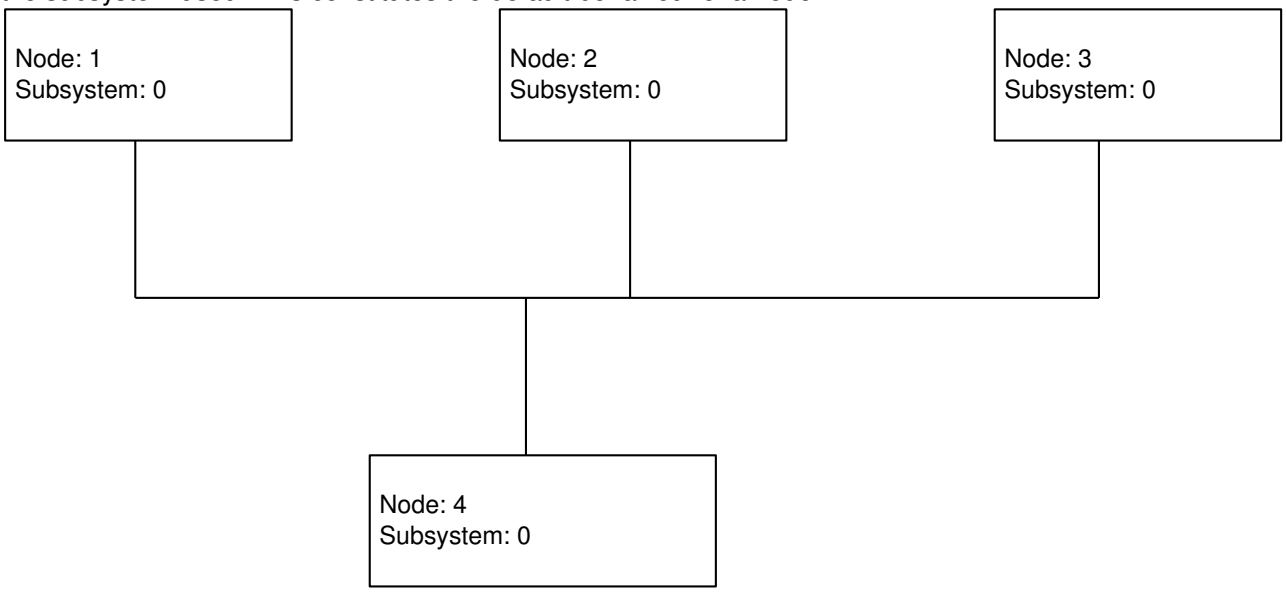


Figure 3: Subsystem differentiation scenario 2.

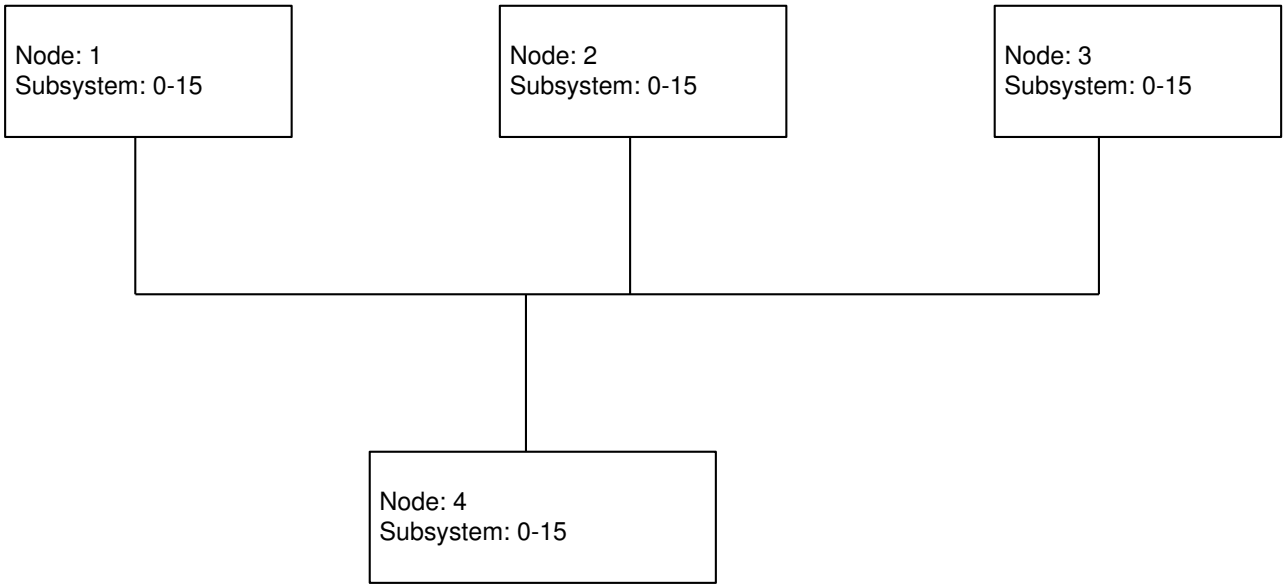


Figure 4: Subsystem differentiation scenario 3. Default behaviour.

When using differentiation of physical nodes via subsystem addressing (as shown in Figure 2) care must be taken if network management is to be used. NMT messages are typically always directed to the network management subsystem – subsystem 15 (See chapter 7).
The first byte after the idle time is handled as the message length in the D-Bus2 specification. If the receiving node detects a message length of zero or one (which is invalid for D-Bus-2.1 and D-Bus-2.2), it must assume that a D-Bus-3 frame is currently sent over the communication line. In this case the D-Bus-2.1/D-Bus-2.2 receiving node must go into a state, where the data on the bus will be ignored, but the line must be monitored to detect the end of the frame by detecting the idle time between two frames. If the end of the frame detected the node can send out after the random waiting time or receive the next frame.

2.4 Idle Recognition

Each communication partner must monitor the bus and is not allowed to transmit anything until idle state is detected[5]. Idle state is detected when no reception is detected for a minimum time since the last byte was received. The minimum waiting time is slightly different from one communication partner to another, as the communication partner's own node address works as a seed in the random generation. For timing issues, please refer to Figure 6 Timing and section 4.3.

$t_{idle,min} > t_{a,max}$ (max interbyte time): minimum time in which the bus must be idle (globally over all communication partners): 10 bit times
To $t_{idle,min}$ a random time is added for each communication partner. For this purpose, the amount of succeeding data collisions can be calculated according to the following table:

Amount of succeeding collisions	Additional waiting time Random time between
0	X + 0-7 bit times
1	X + 0-15 bit times
2	X + 0-31 bit times
3	X + 0-63 bit times
4 and more	X + 0-127 bit times

Table 3: Data Collisions ("x" is a constant time specific to the given communication partner. It is added, if the sent message is not a service message.)

[5] If a communication partner receives a non-broadcast frame (i.e. addressed communication), the acknowledgement should of course be sent without waiting for idle state, as the original sender expects this (the sender would repeat the sent frame after a time-out, if the acknowledgement would follow from the addressed communication partner). In this case the receiver temporarily takes the role of the sender.

The collision counter is reset after successful transmission/reception – but it is unaffected by a negative acknowledgement (acknowledgement busy or acknowledgement wrong/CRC error).

Calculation of the random number (see section 2.5 and 4.6 for a description of data types):

```
static   uint16   randx;

uchar rand127(void) {  // random numbers between 0-127
  return((int)((randx = (randx ^ 11035) + 12345)>>8) & 0x7F);
}
uchar rand7(void)  { return rand127()& 0x07; }
uchar rand15(void) { return rand127()& 0x0F; }
uchar rand31(void) { return rand127()& 0x1F; }
uchar rand63(void) { return rand127()& 0x3F; }

randx = 0x1798+CommunicationPartnerNumber;  // Initialisation
```

2.5 C data types used

For the examples used in c in this document the following types are referred to:

```
typedef unsigned char     uchar;
typedef unsigned int      uint16;
typedef unsigned long     uint32;

typedef uint16 TbusMessageIdentifier;      //the message identifier is an uint16

typedef void (*TbusService)(uchar ucDataLen, uchar *pucData);       //Generic interface for handling message data
typedef void (*TbusConfirmationService)(void);

typedef struct {
  uchar                   ucMessageLength;
  service                 ucTargetAddress;
  TbusMessageIdentifier   tMessageIdentifier;
} TbusIdentifier;

typedef struct {
  TbusIdentifier          tBusIdentifier;
  TbusService             tServiceFunction;
}TbusReceiveObject;               //The table is terminated with a record, where MSB of the MessageLength is set[6].

typedef struct {
  TbusIdentifier          tBusIdentifier;
  uchar                   ucDataLen;
  TbusService             tServiceFunction;
  TbusConfirmationService tConfirmationFunction;
}TbusTransmitObject;
```

[6] Please note that setting the MSB of the message length does not influence the "real" message length, as the effective message length is received from the bus, regardless of the value defined in the table.

3 Physical Layer

The physical layers of the D-Bus-2, D-Bus-2.1 and D-Bus-2.2 are identical.

For information that is more detailed, please refer to the external reference “DBUS Hardware Datasheet” mentioned in chapter 1.3.

In addition, D-Bus-2.2 is supported by DBusCAN chip. For more information please refer to the external reference “DBusCAN System Basic Chip” mentioned in chapter 1.3.

4 Data link Layer

The Data Link Layer format has not changed between D-Bus-2 and D-Bus-2.2.

4.1 Frame Format

All data on the bus is packed into so-called frames. The frame contains Address, Length and Checksum information in addition to the message (data to be transmitted). One acknowledgement belongs to each frame. The acknowledgement is generated by the addressed communication partner (receiver) – except for broadcasts, which are acknowledged by the sender.

	Frame						
<idle>	**MessageLength**	**Target Address**	**Message (Data)**	**CheckSum**	*<gap>*	***Ack***	*<idle>*
	1 Byte	1 Byte	2..n Byte	16 Bit		1 Byte	

Table 4: Frame structure

The minimal length of a useful frame is 6 Byte (MessageLength + TargetAddress + MessageIdentifier + CheckSum), however, also a frame of 4 byte will be recognised as valid (MessageLength = 0).

4.2 Byte Format

UART based transmission consists of data where the bits are packed into bytes. The serial transmission means that one byte is sent after the other. A byte consists of a start bit (marked as S in Figure 5 below) followed by the 8 “useful” bits of the byte (bit0 first, bit1 second, …, bit7 last) and a Stop or End bit (marked as E in the figure below). The time between the bytes within a transmission is called interbyte time, this is described in section 4.3Timing and section 2.4 Idle Recognition.

Figure 5: Byte format

4.3 Timing

As already described in this chapter a data stream consists of data packages separated by a certain period of time. The smallest data packet is the byte (described in the previous section). The separation between two bytes, outlined as t_{ibyte} in Figure 6 is called the interbyte time.

When all data in a frame has been transmitted, this must be acknowledged by the receiver (or the sender when broadcasting). The time between the transmitted data and the acknowledgement is outlined as t_a in Figure 6.

The frame is ended when the acknowledgement has been transmitted. If the sender wants to send another frame, this is only allowed after an idle period of time, indicated as t_{idle} in**Figure 6:** Timing. If the sender would send another frame without waiting, an erroneously programmed communication partner could block the bus. Exceptions may be done for diagnostic purposes, or when a PC wants to update the controller (care must be taken not to block the bus…).

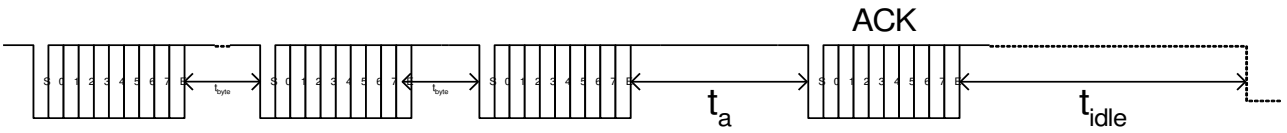


Figure 6: Timing

$t_{a,min}$: minimum time between the end of a frame and the start of the acknowledgement is 0, i.e. an acknowledgement may be transmitted immediately.

$t_{a,max}$: maximum time between the end of a frame and the start of the acknowledgement <= max. interbyte time ($t_{ibyte,max}$). Is this time exceeded all communication partners, upon recognising a timeout, discard the corresponding message.

$t_{ibyte,min}$: 0 Minimum time between two Bytes

$t_{ibyte,max}$: 9 bit times (by 9600 Baud = 937 µs). This value decreases with increased baudrate up to 38400 Baud (by 38400 Baud = 234 µs). At higher baud rates $t_{byte,max}$ remains at 234 µs (considering UART clock tolerance +/- 1.5%, worst case is nominal + 1.5%, which gives $t_{byte,max}$ 230.85 µs) .

Please refer to section 2.4 for a description of t_{idle}.

If there is no pause between the bytes (interbyte time), the distance between the start of two received bytes is 10 bit times, at 9600 Baud this corresponds to 1041.66 µs.

The maximum tolerable pause time, $t_{ibyte,max}$, results in a period between the start of the received bytes of 19 bit times (10 bit data + 9 bit pause time), or 1979.16 µs @ 9600 Baud.

As long as the time between the frame and the corresponding acknowledgement (Acknowledgement time-out) is smaller than the maximum interbyte time, the acknowledgement can be allocated to the correct frame.

4.4 Message Length

As a frame, additionally to the message content (including the message identifier), consists of 4 bytes, the message length can be calculated as:

MessageLength = FrameLength - 4.

For a message with n data byte, the MessageLength is: n+2 (2 is the length of the message identifier).

	MessageLength	**FrameLength**
Min	2	6
Max	255-4	255

Table 5: Message/Frame Length

NB! A MessageLength of 0 is also valid on a D-Bus-2, but a useful frame needs a message identifier (assuming there is more than one known message in the system). A MessageLength of 0 is not any longer valid on the D-Bus-2.1, because the D-Bus-3 starts with a version flag that is always 0. The D-Bus-3 and D-Bus-2.1/D-Bus-2.2 compatible receivers use this to identify a D-Bus-3 frame. A D-Bus-2.x frame starts always with a byte greater or equal 2.
The maximum length should never be longer than the length of the receive buffer specified by the application designer (see chapter 6).

4.5 Target / Source Address

For better reading the values referred to in this section are hexadecimal.

Bit Number:	**Target / Source Address**							
	7	6	5	4	3	2	1	0
	Communication Partner				SubSystem			

Table 6: Target/Source Address

Communication Partner = x	**Value Target / Source Address (Hex)**	**Standard/Application specific**
NODE_0	0? = ALL: Broadcast communication	Standard
NODE_1	1? (e.g. Power module)	Application specific
NODE_2	2? (e.g. Panel module)	
NODE_3	3? (e.g. Sensor)	
…		

Communication Partner = x	Value Target / Source Address (Hex)	Standard/Application specific
NODE_Remote	A? = VirtualProcessorRemoteControl	Standard
NODE_SI	B? = ProcessorSystemInterface	
NODE_PC	C? = PC / Diagnostic Station	
…		Application specific
NODE_F	F? = Reserved for future extensions	Standard

Table 7: CommunicationPartner

Target address 0: All communication partners are addressed. Acknowledgement is made by sender (refer to Chap 4.10 Broadcast Communication).

Target address 1-9 + D-E: Applications specific meaning (no interoperability/compatibility).

Target address A: Virtual Processor, for the RemoteControl.

- The remote extension of the appliance is the communication partner of the SystemInterface (SI). It is a virtual contact of the SI, as it is not known where the remote extension is – whether accommodated under the power module, panel module or another module.

Target address B: SystemInterface (SI).

Target address C: PC / Diagnostic station

Target address F: Reserved for future extensions

SubSystem	Value Target / Source Address (Hex)
SUBSYS_0	?0 = normal addressing (e.g. when no direct addressing is possible, due to unknown subsystem structure of other node)
SUBSYS_1	?1 (e.g. Sound)
SUBSYS_2	?2 (e.g. Display)
…	…
SUBSYS_F	?F (e.g. NMT)

Table 8: SubSystem

Maximum 15 Communication partners can be addressed, each communication partner can address 16 SubSystems.

SubSystem 0 should[7] be used for exchanging service messages. The further differentiation can be used by the application for a direct addressing (e.g. sending a system message from the motor controller subsystem to the display controller subsystem).

SubSystem F is typically reserved for network management messages (see chapter 7).

It is possible that a communication partner reacts to more than one communication partner address (alias addressing). In this case the subsystems of the alias address will be 8-bit long.

Example:

A power unit has native address 1 in addition to an alias address A (remote control extension). To differentiate subsystem 0 of native address 1 from the alias-addressed subsystem 0 it is necessary to consider the full eight-bit value of this subsystem.

This node could have the following subsystems: {0, 1, 5, 9, A, A0, A3, A9, C, F}

[7] Exceptions are allowed, if e.g. physical nodes are differentiated via subsystem addressing (see **Figure 2**, **Figure 3** and **Figure 4** in section 2.3).

4.6 Data Types

The data in the messages (see Table 4) referred to in this document are of the following types:

Name	Description
uchar	Unsigned integer with length 1 byte. Range: 0..255
uint16	Unsigned integer with length 2 byte. Range 0..65535
uint32	Unsigned integer with length 4 byte. Range 0..4294967295 (0..2^{32}-1)
schar	Signed integer with length 1 byte. Range: -128..127
int16	Signed integer with length 2 byte. Range -32768..32767
int32	Signed integer with length 4 byte. Range –2147483648.. 2147483647 (-2^{31}.. 2^{31}-1)

Table 9: Data Types

The message format is Big-Endian (a message containing an unsigned integer is transmitted with the high byte first) – for signed data types: 2's complement.

4.7 Check Sum – Cyclic Redundancy Check (CRC)

Check sum - CRC	Method
16 Bit CRC	Xmodem 16bit (polynomial 1021h), Start=00

Table 10: CRC

The check sum covers the length, address and message content.

Frame			
MessageLength	**Target Address**	**Message (Data)**	**CheckSum**
1 Byte	1 Byte	2..n Byte Message ID 16 bit | Data n-2	16 Bit

CRC Calculation

Msg.Len

Table 11: D-Bus-2 Frame

For better performance the CRC calculation method used by D-Bus-1 has been replaced by a 16-bit CRC calculation. This makes D-Bus-2 being incompatible to D-Bus-1.

Upon creating a CRC, the polynomial is often referred to. The polynomial should be interpreted bit wise: x^4 refers to bit 4. A polynomial $x^3 + x^7$ can be represented as 88h (10001000b): $1*x^7 + 0*x^6 + 0*x^5 + 0*x^4 + 1*x^3 + 0*x^2 + 0*x^1 + 0*x^0$

16 bit CRC detects the following transmission errors:

- Single bit errors
- Two single bit errors in data frames shorter than $2^{(n-1)}$ bit (16-bit CRC: n=16: 32768 bit = 4096 Byte)
- All uneven counts of bit errors
- All error bundles (bursts) which have shorter or equal length to the CRC

Error bundles longer than the CRC (≥ 9 bit) will not be recognised by a probability of $0.5^{(16-1)}$ = 1 out of 32768.

The rest-error probability is 0.5^{16}, assumed even distribution.

Calculating a 16 bit XMODEM-CRC can be done with the following program downloaded from the Internet:

```
/* (Extracted from a document edited by Chuck Forsberg of
 * Omen Technology in Portland, Oregon)
 *
 * This function calculates the CRC used by the XMODEM/CRC Protocol
 * The first argument is a pointer to the message block.
 * The second argument is the number of bytes in the message block.
 * The function returns an integer which contains the CRC.
 * The low order 16 bits are the coefficients of the CRC.
 */
```

```
int16 calcrc(ptr, count)
int8 *ptr;
int16 count;
{
    int16 crc, i;

    crc = 0;

    while (--count >= 0) {
        crc = crc ^ (int)*ptr++ << 8;

        for (i = 0; i < 8; ++i)
            if (crc & 0x8000)
                crc = crc << 1 ^ 0x1021;
            else
                crc = crc << 1;
        }
    return (crc & 0xFFFF);
}
```

4.8 Acknowledgement

Bit Number:	**Acknowledgement**							
	7	6	5	4	3	2	1	0
	Communication Partner				Acknowledgement Type			

Table 12: Acknowledgement

The acknowledgement is normally (for broadcast messages, see section 4.10) created (and appended to the message) by the addressed communication partner. The Communication Partner, referred to by the acknowledgement, refers to the addressed node. If a communication partner has alias addresses (e.g. communication partner 1 responds also to address A), it is expected that the address used in the target address is confirmed in the acknowledgement. Broadcast messages will of course be acknowledged by using the "native" address of the sending node.

Ack Type	Value (Hex)	Comment
ACK_OK	?A	After successful reception (CRC Check included)
ACK_BUSY	?3	e.g. by buffer overflow or busy system
ACK_WRONG	?7	Error by reception (e.g. wrong CRC)

Table 13: Acknowledgement type

The acknowledgement must follow within $t_{idle,min}$, otherwise the frame will be repeated (sender believes that the frame was not received if the acknowledgement does not appear before idle state is recognised).

4.9 Addressed Communication

By the addressed communication the sender waits for the acknowledgement of the sent frame. Only one node is addressed, hence this node has to acknowledge.

4.10 Broadcast Communication

By broadcasts the sender cannot expect any communication partner to acknowledge, so the acknowledgement is sent by the sender itself (naturally only after having read back the correct frame from the bus, i.e. no data collision). The sender does not get any direct feedback whether the message has been received by any other nodes.

If the sender of a broadcast message detects an error in the bit stream sent on the bus, an acknowledgement will not be sent. The receivers will detect a time-out, resulting in the erroneous message being discarded.

4.11 Interface Data Link Layer

The normal usage of the D-Bus-2 is to use service functions for preparing data for transmission and for saving received data. However, it is also possible to initiate a message transmission by directly transmitting a message structure via a data link layer interface function.

When a message is received this will be distributed to the subsystem corresponding to the lower nibble of the target address byte.

5 Presentation Layer: Data Format

The data sent from one communication partner to another is transmitted in messages (which are parts of the frames). All messages marked as deprecated below **(in chapter 5 and all subchapters)** are still supported for software, which is older, than V1.8 of Dbus 2.2 specification. All deprecated messages will eventually be replaced by other messages, which have been or will be specified following concepts that are more modern. In the meantime, both old and new concepts will exist in parallel for several years.

5.1 Predefined Service Messages

Predefined service messages are defined in section 5.2.2 (common service messages defined for all communication partners), in section 5.2.3 (optional service messages defined only for certain communication partners), in section 5.2.13 (Update Service Messages) and in section 5.2.14 (Production Service Messages). These messages are mostly used for diagnostic or maintenance purposes and are processed by the application layer. Each slave on the bus can be addressed with predefined service messages by only one master at a time.

Service messages, for which a response is expected, are only to be used in the frame of a directed communication (i.e. not as broadcasts), unless it is explicitly stated otherwise for the given message (Wakeup Sent Request, Power Fail Message).

The following messages are mandatory for all DBus nodes:

- Read Memory Request
- Read Memory Response
- Identity Request
- Identity Response

The following messages are optional and may be implemented, depending on the corresponding application needs, power management requirements, SW and HW configuration (Red messages have become redundant due to introduction of Update Service Messages. Violet messages will eventually become redundant, because NMT is being replaced by solutions that are more modern.):

- Write Memory Request (message used for writing memory)
- GoOffline (message to communication partner that an update will follow)
- Reset (reset for the bootloader – specification of waiting time)
- Set Memory Module (to make it possible to distinguish several memory modules)
- Read Memory Request32 (for 32 bit addressing)
- Read Memory Response32 (for 32 bit addressing)
- Write Request32 (for 32 bit addressing)
- Get Block Size Request (requests the maximum data length for read/write commands)
- Block Size Response (the maximum data length restricted by buffer or EEPROM)
- Erase Block Request (erases a block – typically necessary before writing flash memory.)
- Erase Block Response (erase finished successfully)
- Start Stream Mode (start flag for update purposes)
- End Stream Mode (stop flag for update purposes)
- Update Status Request (for checking whether update was successful)
- Update Status Response (status of update: Successful / unsuccessful)
- Update Status Reset (reset update status flag)
- Ping (message to test, if node communicates in the given baud rate)
- Wakeup Sent Request (message to test, by whom a 15ms wake up has been sent (6.1.3))
- Wakeup Sent Response (message to confirm sending of wake up signal)
- Ecu Unique Id Read Request (message to request Id, which is unique to the electronic)
- Ecu Unique Id Read Response (message to send Id, which is unique to the electronic)
- Power Fail Message (message to signal power supply problems)

- Power Resurge Message (message to signal power supply recovery event)
- Power Management Wakeup (message to wake up the Node over DBusCAN chip)
- Power Management Wakeup Complete (message to cancel wake up process of Node over DbusCAN chip)
- Power Management Node Reset (message to trigger power-on-reset of Node over DBusCAN chip)

The following messages are for update purposes:

- Silent Mode Transition Request
- Silent Mode Transition Response
- Update Mode Verify Request
- Update Mode Verify Response
- Update Mode Transition Request
- Baud Rate Verify Request
- Baud Rate Verify Response
- Baud Rate Transition Request
- Baud Rate Trigger Request
- Reset Trigger Request
- HSI Protocol Request
- HSI Protocol Response
- Return from Silent Mode Request
- Return from Silent Mode Response
- ECU Config Read Request
- ECU Config Read Response
- ECU Software Submodule Read Request
- ECU Software Submodule Read Response

The following messages are for production purposes:

- Write Ecu Config Hw Request
- Write Ecu Config Hw Response
- Write Tracing Id Request
- Write Tracing Id Response
- Write Production Time Request
- Production Time Reponse
- Read Production Time Request
- Write Teststate Request
- Teststate Repaircount Reponse
- Read Teststate Repaircount Request
- Write Appliance Data Request
- Write Appliance Data Response
- Read Appliance Data Request
- Read Appliance Data Response
- Factory Reset

5.2 Messages

Different kinds of messages are defined. The following values of the message identifiers are hexadecimal.

Service Messages (F000h-FFFFh):
These messages do not serve the normal information exchange between the different communication partners, as they are not used in the normal operation of the appliance. Service messages allow direct read and write access.

Network Management Messages (E000h-EFFFh): DEPRECATED, supported for software older than this remark
These messages control the network.

After Sales Service Communication objects (9000h-9FFFh) are described by Reference 3. Test messages are used for automatic testing in combination with a test server running on the target.

Remote Control Messages (I@)(8000h-8FFFh): DEPRECATED, supported for software older than this remark
These messages are used in the normal operation of the appliance, when a SystemInterface is integrated, and this possibly can communicate with a residential gateway over e.g. power line or RF. By using these messages the user is able to remote control / monitor the appliance.

System Messages (0000h-7FFFh):
These messages are used in the normal operation of the appliance. Different communication partners exchange data by using system messages. When network management (see chapter 7) is implemented, the system messages are processed only when allowed by the network management.

Service Messages and network management messages are mostly application independent. Remote Control Messages are application independent in form and definition, but application specific regarding usage and implementation[8]. System Messages are application specific.

For all service messages and system messages explicit type definitions are declared. The type definitions found in this specification are used to illustrate the data structures of the defined messages. Due to problems related to alignment and byte order of values, which are not 8-bit size, it is recommended to use an explicit byte array rather than a data structure.

F000 - FFFF	Service Messages (F800 - F9FF reserved for proprietary messages)
FD01 – FD2F	Drives Control Diagnostic Messages
E000 - EBFF	NMT Messages DEPRECATED, supported for software older than this remark
EC00 - ECFF	Power Management Node Messages
ED00 - EFFF	NMT Messages DEPRECATED, supported for software older than this remark
D100 - DFFF	Undefined
D000 – D0FF	Functional Safety Messages
BE00 - CFFF	Reserved for Smart Sensor Bus (SSB) Messages
BC00 - BDFF	Common Components Messages

[8] Example: A dishwasher does not use spinning speed, hence this object is not implemented. Nevertheless this object may be defined, to avoid the identifier from being used for the dishwasher.

BA00 - BBFF	Touch Messages
B9FE – B9FF	Microcontroller Debug Messages (Specification is contained in uc-Framework)
B9F0 - B9FC	Sigma Data Messages (Reference [20] in Table 2: Coexisting Documents)
B9E0 – B9EF	SMM Test Server Touch Messages
B980 – B98F	Generic Appliance Communication Messages
B990 – B9DF	Undefined
B900 – B97F	Common Bluetooth Module Messages
A300 – B8FF	Undefined
A200 – A2FF	SMM Device Calibration and Test Messages
A000 – A1FF	System Master Messages
9000 - 9FFF	After Sales Service Communication objects and Test Messages (automatic tests)
8000 - 8FFF	Remote Control Messages DEPRECATED, supported for software older than this remark
0000 - 7FFF	System Messages

Table 14: Message Distribution.

5.2.1 Reserved Service Messages

A range of service messages is reserved for different users to be able to define their own service messages. These service messages will not be part of this specification, nor is the interworking between different systems considered, as different systems may use an identifier in this range in different ways.

The specified range is from 0xF800 till 0xF9FF.

5.2.2 Common Service Messages

The service messages defined in this sub chapter must be implemented. A request must be answered within 100 ms to avoid any deadlock due to repeated requests. Each communication partner will, however, answer as soon as possible.

A module may be set (using the optional service message SetModule) to distinguish among different memory ranges (e.g. external EEPROM, RAM/ROM and internal EEPROM). By default, (always when SetModule is not implemented) the memory module is set to 0.

5.2.2.1 Msg ReadRequest

This message is used for the communication with a pc (diagnostic station). The message direction is normally from pc to device. The Source Address is the address of the sender of this message.

Message Identifier	Msg Parameter1	Msg Parameter2	Msg Parameter3	Msg Parameter4
	Source Address	Bytes to Read	Address High	Address Low
F000h MSG_READREQUEST	Node/SubSys	<1..246>	<any addr high>	<any addr low>

Length: 6 Byte (optionally 7)
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;     //Message identifier
  uchar ucSource;         //Sender address - normally 0xC0 (PC)
  uchar ucNumberOfBytes;  //DataLength to read
  uchar ucAddrHigh;       //Address[1] - for Read access
  uchar ucAddrLow;        //Address[0] - for Read access
} TreadByte;
Optionally another parameter (1 byte) may be added after the Address: ucMemoryModule. This allows the
usage of more than one master (using different memory modules).
```

5.2.2.2 Msg ReadResponse

This message is used for the communication with a pc (diagnostic station). Message direction normally from device to PC. Source Address is the address of the sender of this message.

Message Identifier	Msg Parameter1	Msg Parameter2	Msg Parameter3…
	Source Address	# of Bytes	Data[n]
F100h MSG_READBYTERESP	Node / SubSys	<1...246>	<data[n]>

Length: 5-250 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;     //Message identifier
  uchar ucSource;                 //Address, with which sender has been addressed in last received message
  uchar ucNumberOfBytes;          //DataLength for read access
  uchar aucData[MAX_LENGTH];      //Data to read
} TreadByteResponse;
```

Note: depending on the output buffer size the DataLength value may be different from the DataLength sent to the target in ReadRequest.

5.2.2.3 Msg IdentityRequest

Request of the ID_Blocks. Message direction only from PC to device.

Message Identifier	Msg Parameter1
	Source Address
FF00h MSG_IDENTITY	Node / SubSys

Length: 3 Byte (optionally 4)
Type:

```
typedef struct {
  TbusMessageIdentifier  tMsg;           //Message identifier
  uchar                  ucSource;       //Sender address
} TidentityAdressRequest;
Optionally a second parameter (1 byte) may be added after the Source Address: ucMemoryModule. This allows
the usage of more than one master (using different memory modules).
```

Note: If the ID String is not in MemoryModule 0 then the SetMemoryModule command may be called before the IdentityRequest.

5.2.2.4 Msg IdentityResponse

Feedback to the identity request is a pointer to the Id-Block in question. Message direction only from device to pc.

The returned 16-bit or 32-bit value represents a pointer to the first character of the identification string defined in chapter 2 (Allgemeiner Aufbau des ID-Sting's – Reference [2] in Table 2: Coexisting Documents) in the identification of hardware and software components. The string is read byte by byte, until it is ended by a double 00h.

This message typically returns a 16-bit address (of the first ID-string character), when the ID-string is situated in a memory, which can be addressed by using 16-bit. When the ID-string is placed in an area, which needs more than 16-bit for addressing, a 32-bit value is returned (BigEndian format).

Message Identifier	Msg Parameter1 Sender Address	Msg Parameter2 Module	Msg Parameter3 Address of ID-Block
FE00h MSG_IDENTITY_RESPONSE	Node / SubSys	Memory Module	addrIdBlock[2/4]

Length: 6/8 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;//Message identifier
  uchar    ucSender;          //Address, with which sender has been addressed in last received message
  uchar    ucModule;          //The module, to which the address referred to belongs.
  uint16  uiAddrIdBlock;      //Address of the first Byte of the identification string - Big-Endian Format
  [uint32 uiAddrIdBlock;    //Address of the first Byte of the identification string - Big-Endian Format]
} TidentifyAddressResponse;
```

5.2.3 Optional Service Messages

Service messages mentioned below may be implemented, depending on the corresponding application needs, power management requirements, SW and HW configuration.

5.2.3.1 Msg Ping

This message is used to test, whether a given node has a certain baud rate.

Message Identifier
F080h MSG_PING

Length: 2 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;     //Message identifier
} TreadByte;
```

5.2.3.2 Msg WriteByte

This message is used for the communication with a pc (diagnostic station). Message direction normally from pc to device.

Message Identifier	Msg Parameter1 # of Bytes to Write	Msg Parameter2 Address High	Msg Parameter3 Address Low	Msg P4…. Data to write
F200h MSG_WRITEBYTE	<1…246>	<any addr high>	<any addr low>	<data[n]>

Length: 6-251 Byte (optionally: 7-251 bytes)
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;     //Message identifier
  uchar ucNumberOfBytes; //DataLength for writing – normally 1 or 2
  uchar ucAddrHigh;       //Address[1] – for write access
  uchar ucAddrLow;        //Address[0] – for write access
  uchar aucData[MAX_LENGTH];
} TwriteByte;
```

Optionally another parameter (1 byte) may be added after the last data byte: ucMemoryModule. This allows the usage of more than one master (using different memory modules).

5.2.3.3 Msg GoOffline

Bootloader message

This message is used for the communication with a pc (diagnostic station), when a bootloader is used. Message direction from pc to device.

For a bootloader to be able to talk to only one communication partner the others must go offline.

The message should be sent as a broadcast to announce all nodes that an update (of one communication partner) is following. Please note that in case NMT is active, the message NMT-ShutDown might be needed prior to starting the update, refer to section 7.6.1.7.

Message Identifier	Msg Parameter1	Msg Parameter2	Msg Parameter3
	Node2Update	ResetDelay (*10 ms)	OfflineDelay (*2 s)
FDFFh MSG_GO_OFFLINE	Node/SubSys	<0...255>	<0…255>

Length: 5 Byte (optionally: 6 Byte)
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;     //Message identifier
  uchar ucNode2update;    //The communication partner which will be updated via the bootloader
  uchar ucResetDelay;     //The update-partner waits between 0 and 2550ms before reset is executed
  uchar ucOfflineDelay;   //After 0-510 seconds the offline communication partners execute reset
} TgoOffline;
```

The message has two meanings:

1. For the communication partner (only one) which is being addressed by Message parameter 1: After 0-2550 ms (message parameter 2) delay a reset is executed. Following, this controller will be updated via the bootloader. After the update has finished the bootloader forces the controller to execute a reset.
2. For all other communication partners (not addressed by message parameter 1): No action is taken to any commands being received on the bus according to the OfflineDelay. If OfflineDelay is set to 0, the delay is infinite, i.e. the nodes will stay offline until a reset has been forced (e.g. by removing the power). These communication partners will not transmit anything on the bus, until a reset has been executed after the OfflineDelay. More specifically, this might be either a hardware reset or a software reset triggred by an administrative protocol.

Whether the controller needs to enter a special state before the delay time is application specific and must be determined for each system (e.g. whether the door has to be locked and the heating switched off).

5.2.3.4 Msg ResetExecute deprecated (new: Common Components Messages, Update Service Messages)

Message used e.g. by Bootloader

This message is used for the communication with a pc (diagnostic station), when a bootloader is used. Message direction normally from pc to device. The message is not defined as an NMT-message, as this message can be needed also for controllers not implementing the NMT.

Message Identifier	Msg Parameter1
	ResetDelay (* 10 ms)
FD00h MSG_RESET_EXECUTE	<0..255>

Length: 3 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;     //Message identifier
  uchar           ucDelay;        //Delay between 0 and 2550 ms before reset is executed
} TresetExecute;
```

5.2.3.5 Msg SetMemoryModule

This message is used for the communication with a pc (diagnostic station). Message direction normally is from PC to device.

The MemoryModule will not be reset upon communication time-out, only after explicitly receiving a new SetMemoryModule, or upon a controller reset.

Message Identifier	Msg Parameter1
	Memory Module
F300h MSG_SETMODULE	<0..255>

Length: 3 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier  tMsg;                  //Message identifier
  uchar                  ucMemoryModule;        //Target memory module, e.g. 0 for Processor RAM/ROM
} TsetMemoryModule;
```

The SetMemoryModule command is independent of a possibly used optional, additional byte (ucMemoryModule), i.e. if SetMemoryModule 1 is received, then a ReadRequest using a different memory module does not affect this memory module beyond the read response.

5.2.3.6 Msg ReadRequest32

This message is used for the communication with a pc (diagnostic station). Message direction is normally from PC to device. Read request with 4 Byte addressing (32 Bit addressing).

Message Identifier	Msg Parameter1	Msg Parameter2	Msg Parameter3-5
	Source Address	# of Bytes to Read	Address (Big-Endian)
F001h MSG_READREQUEST32	Node/SubSys	<1..244>	<any addr[4]>

Length: 8 Byte (optionally 9)
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;     //Message identifier
  uchar ucSource;                 //Sender address - normally 0xC0 (PC)
  uchar ucNumberOfBytes; //DataLength to be read - normally 1 or 2
  uchar ucAddress[4];     //4 Byte Addressing for Read access - Big-Endian Format
} TreadByte32;
```

Optionally another parameter (1 byte) may be added after the Address: ucMemoryModule. This allows the usage of more than one master (using different memory modules).

5.2.3.7 Msg ReadResponse32[9]

This message is used for the communication with a pc (diagnostic station). Message direction is normally from device to PC. ReadResponse32 to be used when 4-byte addressing was used in the request (ReadRequest32).

Message Identifier	Msg Parameter1	Msg Parameter2	Msg Parameter3…
	Source Address	# of Bytes	Data[n]
F101h MSG_READBYTERESP32	Node/SubSys	<1...244>	<data[n]>

Length: 5-250 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;     //Message identifier
  uchar ucSource;                 //Address, with which sender has been addressed in last received message
  uchar ucNumberOfBytes;          //DataLength to be read - normally 1 or 2
  uchar aucData[MAX_LENGTH];      //Data to be read
} TreadByteResponse32;
```

Note: depending on the output buffer size the DataLength value may be different from the DataLength sent to the target in ReadRequest.

[9] NB! This message is the same as ReadResponse (16 Bit addressing), but it is beneficial to define an explicit message (=own Identifier) for the 32 Bit addressing response.

5.2.3.8 Msg WriteByte32

This message is used for the communication with a pc (diagnostic station). Message direction is normally from PC to device. Write request with 4-byte addressing.

Message Identifier	Msg Parameter1	Msg Parameter2-4	Msg Parameter 5…
	# of Bytes to Write	Address (Big-Endian)	Data to write
F201h MSG_WRITEBYTE32	<1…244>	<any addr[4]>	<data[n]>

Length: 8-251 Byte (optionally 9-251 Byte)
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;     //Message identifier
  uchar ucNumberOfBytes;          //DataLength to be written
  uchar ucAddress[4];             //4 Byte addressing for write access - Big-Endian Format
  uchar aucData[MAX_LENGTH];      //Data to be written
} TwriteByte32;
```

Optionally another parameter (1 byte) may be added after the last data byte: ucMemoryModule. This allows the usage of more than one master (using different memory modules).

5.2.3.9 Msg BlockSizeRequest deprecated (new: Update Service Messages)

Request the block size of the addressed communication partner. Message direction is normally from PC to device.

Message Identifier	Msg Parameter1
	Source Address
F400h MSG_GET_BLOCK_SIZE	Node / SubSys

Length: 3 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;     //Message identifier
  uchar ucSource;                 //Sender address
} TblockSizeRequest;
```

5.2.3.10 Msg BlockSizeResponse deprecated (new: Update Service Messages)

Feedback to the block size request is the optimal/maximum block size, which this communication partner can handle (due to buffer size and writing to EEPROM). Message direction normally from device to pc – broadcast may be used. A second optional databyte will be used for controllers, which need blocks to be erased before they can be written. This byte is the exponent *n* (power of 2) of the block size.

Message Identifier	Msg Parameter1	Msg Parameter2
	Block size Write	Block size Erase
F500h MSG_BLOCK_SIZE_RESPONSE	1..128	2^n

Length: 3/4 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;     //Message identifier
  uchar ucBlockSizeWrite;         //Block size of of the requested communication partner
  <uchar ucBlockSizeEraseExp;     //Optional Parameter for exponent of block size for erasing.>
} TblockSizeResponse;
```

5.2.3.11 Msg EraseBlockRequest deprecated (new: Update Service Messages)

Command normally from PC to device to erase a block of memory. It is expected that set memory module is used before this command is used. It is only needed to use this command before writing memory of a module, which block size response has two elements (last element is optional, and reports the size of an erasable block).

Message Identifier	Msg Parameter1	Msg Parameter 2	Msg Parameter 3
	Source Address	Start Address	End Address
F600h MSG_ERASE_BLOCK_REQUEST	Node / SubSys	<0…FFFFh >	<0…FFFFh>

Length: 7 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;     //Message identifier
  uchar ucSource;                 //Sender address
  uint16 uiStartAddress; //Start Address of block to be erased (relative to current memory module).
  uint16 uiEndAddress;   //End Address of block to be erased (relative to current memory module).
} TeraseBlockRequest;
```

5.2.3.12 Msg EraseBlockResponse deprecated (new: Update Service Messages)

Response from controller, when the requested block has been successfully erased. Please note that the controller may be unable to communicate (generate layer 2 acknowledgement) during the erasing procedure. This message is a feedback that the procedure of erasing the block(s) has ended successfully – normal communication is possible again.

Message Identifier	Msg Parameter 1 EraseStatus
F700h MSG_ERASE_BLOCK_RESPONSE	<0 (error) / 1 (block erased)>

Length: 3 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;     //Message identifier
  bool   bEraseStatus;            //1 = successfully erased the block, 0 = error
} TeraseBlockRequest;
```

5.2.3.13 Msg StartStreamMode deprecated (new: Update Service Messages)

To assure data consistency it is important to have the possibility to tell the controller that an update (or writing of a block) is following. Message direction is only from PC to device. See also description of Msg End Stream Mode for an explanation of use.

Message Identifier
F301h MSG_START_STREAM_MODE

Length: 2 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;     //Message identifier
} TstartStreamMode;
```

5.2.3.14 Msg EndStreamMode deprecated (new: Update Service Messages)

This is the end of the stream mode (e.g. update finished).

To make an update of a controller, the following message sequence is expected:

- Start Stream Mode initiates the update procedure
- The block size to be used is assumed known (either implicitly, or by exchange of Block Size Request + Block Size Response).
- Message WriteByte or WriteByte32 is used iteratively until the complete segment has been transmitted
- Message ReadByte or ReadByte32 can be used in the same way as WriteByte/WriteByte32 for verification of data content in a memory module.
- End Stream Mode ends the update procedure, and flags a successful update completed.

Message direction is only from PC to device.

Message Identifier
F302h MSG_END_STREAM_MODE

Length: 2 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;        //Message identifier
} TendStreamMode;
```

5.2.3.15 Msg UpdateStatusRequest deprecated (new: Update Service Messages)

Message direction is only from PC to device.

Message Identifier	Msg Parameter1 Source Address
F303h MSG_UPDATE_STATUS_REQUEST	Node / SubSys

Length: 3 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;     //Message identifier
  uchar ucSource;                 //Sender Address
} TupdateStatusRequest;
```

5.2.3.16 Msg UpdateStatusResponse deprecated (new: Update Service Messages)

This is the confirmation from the device, which was updated via Stream Mode, whether the process finished successfully. For a stream mode reading process this will, of course, be ok. For a Stream Mode Write process it is important that the device verifies the written bytes. This message informs the pc that the writing procedure finished successfully (or not) – it is not necessary to send all written bytes back over the bus, it is sufficient when the device always controls that the written bytes correspond to the bytes received via the bus.

Message direction is only from device to PC.

Message Identifier	Msg Parameter 1 Stream Mode Status
F304h MSG_UPDATE_STATUS_RESPONSE	<0 (error) / 1 (OK)>

Length: 3 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier  tMsg;    //Message identifier
  bool                   bOk;     //Flag telling that the stream mode process was finished successfully.
} TupdateStatusResponse;
```

5.2.3.17 Msg UpdateStatusReset deprecated (new: Update Service Messages)

This message is used for resetting the flag used by the message 5.2.3.16 **Msg UpdateStatusResponse**. Message direction is only from PC to device.

Message Identifier
F305h MSG_UPDATE_STATUS_RESET

Length: 2 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;         //Message identifier
} TupdateStatusReset;
```

5.2.3.18 Msg WakeupSentRequest

This message may be issued by a node, which has been woken up by a 15ms (10-20ms) break/Wakeup signal. The purpose is to find out, if the node, that received the signal, is to be woken up and by which other participant. The timeout for a response (chapter 5.2.3.19) is application specific. If no response (chapter 5.2.3.19) is received, the request is to be repeated up to three times. If the repetitions are not answered as well, a node that has been woken up is to return to sleep mode.

Nodes that already have been awake, also may sent this message after receiving a 15ms break. In that case, a response (chapter 5.2.3.19) is to be received as a confirmation, that the present state is still correct.

As the message is to be sent as a broadcast, there might be no repetitions, in case of a collision.

Message Identifier	Msg Parameter1 Source Address
F306h MSG_WAKEUP_SENT_REQUEST	Node / SubSys

Length: 3 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;         //Message identifier
  uchar ucSource;                     //Sender address
} TwakeupSentRequest;
```

More information, as well as a graphical scheme, can be found in chapter 6.1.3.

5.2.3.19 Msg WakeupSentResponse

This message is to be issued as a response to the WakeupSentRequest, by the node that did sent a 15ms (10-20ms) Wakeup signal, only if the node issuing the WakeupSentRequest was supposed to be woken up or already awake.

Message Identifier	Msg Parameter1 Source Address
F307h MSG_WAKEUP_SENT_RESPONSE	Node / SubSys

Length: 3 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg; //Message identifier
  uchar ucSource;         //Sender Address, user decides, which of the implemented subsystems to put here

} TwakeupSentResponse;
```

More information, as well as a graphical scheme, can be found in chapter 6.1.3.

5.2.3.20 Msg Ecu Unique Id Read Request

This message may be used to request an identifier, which is unique to the given electronic board, from the addressed node. If a Unique Id is contained on the electronic, the Unique Id must be send in the response. Typically, the Tracing ID (see in chapter for “Production Service Messages”) with a length of 29 Bytes is read here, but it depends on project specific characteristics.

Message Identifier	Msg Parameter1 Source Address
F308h MSG_ECU_UNIQUE_ID_READ_REQUEST	Node / SubSys

Length: 3 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg; //Message identifier
  uchar ucSource;         //Sender Address, user decides, which of the implemented subsystems to put here
} TuniqueIdReadRequest;
```

5.2.3.21 Msg Ecu Unique Id Read Response

This message delivers a unique identifier in response to the "Msg Unique Id Read Request".

Message Identifier	Msg Param1 Source Address	Msg Param2 Status	Msg Param 3 Length	Msg Param4 Bytes
F309h MSG_ECU_UNIQUE_ID_READ_RESPONSE	Node / SubSys	0-OK 1-Update Mode Active 32-Error 33-Unsupported	Length of Id String	Id String (BCD Coded, without zero termination. Each character is one byte in message, with upper nibble having value zero.)

Length: 5-35Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg //message identifier
  uchar ucSource;         //Sender Address, user decides, which of the implemented subsystems to put here
  uchar status;
  uchar length;
  uchar bytes[0 - 30];
} TuniqueIdReadResponse;
```

If the Unique id is encoded as ASCII (or any other non BCD encoding), it will be transmitted as BCD nevertheless. In case any other Unique ID, than the 29 byte Tracing ID (see in chapter for "Production Service Messages") is read, it is the task of the given project to interpret the data read here correctly.

5.2.3.22 Power Fail Message

This message is there for devices in order to be able to bridge short periods (up to 0.5 seconds) in which no power is being supplied. The node, which detected the power fail, must switch off all its' power comsuming sources and send the Power Fail Message to do inform the other nodes to do the same. **The message is to be sent as a broadcast.**

The Power Fail Message has the highest priority, both on the sending and on the receiving side.

Message Identifier
F310h MSG_POWER_FAIL

Length: 2Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg //message identifier
} TpowerFailMessage;
```

5.2.3.23 Power Resurge Message

This message is used to inform other participants that problems with power supply, reported by MSG_POWER_FAIL, have ended. **This message is to be sent as a broadcast.**

Message Identifier	Msg Param1 Source Address
F311h MSG_POWER_RESURGE	Node / SubSys

Length: 3Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg //message identifier
  uchar ucSource; //Sender Address, user decides, which of the implemented subsystems to put here
} TpowerResurgeMessage;
```

5.2.3.24 Power Management Wakeup

This message is used to wake up addressed Node over DBusCAN chip when in advanced power management mode. The message ID and recipient address is automatically recognized and acknowledged by DBusCAN chip and NODE is

to be woken up. For more information please refer to the external reference “DBusCAN System Basic Chip” mentioned in chapter 1.3.

Message Identifier
F320h MSG_POWER_MGMT_WAKEUP

Length: 2Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg //message identifier
} TpowerMgmtWakeup;
```

5.2.3.25 Power Management Wakeup Complete

This message is used to signalize to addressed Node over DBusCAN chip that wakeup process started by WakeUp / Break pulse is complete (no power mode change is required for addressed node). The message ID and recipient address is automatically recognized and acknowledged by DBusCAN chip. For more information please refer to the external reference “DBusCAN System Basic Chip” mentioned in chapter 1.3.

Message Identifier
F322h MSG_POWER_MGMT_WAKEUP_COMPLETE

Length: 2Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg //message identifier
} TpowerMgmtWakeupComplete;
```

5.2.3.26 Power Management Node Reset

This message is used to RESET addressed Node over DBusCAN chip when in advanced power management mode. The message ID and recipient address is automatically recognized and acknowledged by DBusCAN chip. The connected NODE is reset by switching OFF and ON supply voltage and triggering RESET signal of ECU. For more information please refer to the external reference “DBusCAN System Basic Chip” mentioned in chapter 1.3.

Message Identifier
F324h MSG_POWER_MGMT_NODE_RESET

Length: 2Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg //message identifier
} TpowerMgmtNodeReset;
```

5.2.4 NMT Messages deprecated (new: Common Components Messages)

See chapter 7 Network Management (NMT) for a description of these messages.

See chapter 7 for messages related to sleep mode and wake-up.

5.2.5 After Sales Service Communication objects and Test messages

Messages with identifiers between 9000h-9FFFh are reserved for After Sales Service Communication objects and Test messages. After Sales Services Communication objects are described by Reference [12] in Table 2: Coexisting Documents. Test messages are used in combination with a test server, which is running directly on the target, which will

be automatically tested by specially prepared tests. For further information, please refer to reference **Fehler! Verweisquelle konnte nicht gefunden werden.**].

5.2.6 RemoteControl Messages deprecated (new: Technologies, that are not UART protocols, Update Service Messages)

Messages with identifiers between 8000h-8FFFh are reserved for Internet@appliances (RemoteControl). These messages are further defined in separate specification documents (Reference [1] in Table 2: Coexisting Documents).

5.2.7 System Messages

Messages with identifiers between 0000h-7FFFh are meant for data exchange between communication partners during normal operation. These messages are defined in separate application specific documents.

5.2.7.1 Message

Message Identifier
0000h, …, MSG_MESSAGE 7FFFh

Length: 2+n Byte
Type:

```
typedef struct {
TbusMessageIdentifier tMsg;          //Message identifier
…
} Tmsg;
```

5.2.8 Touch Messages

This kind of messages is intended to be used for controlling the behavior of GUIs in a generic way.

5.2.9 Functional Safety Messages

Those messages are to control functionalities, which only are safe, if certain start conditions will be fulfilled.

5.2.10 Common Components Messages

Those messages are for usage by hardware-independent libraries sufficiently generic to be used by all projects using Dbus2, where a master either is controlling slaves or gathering information from them.

5.2.11 System Master Messages

Those messages are to be used for the production and maintenance of the “System Master”, a big controller with a Linux operating system.

5.2.12 Other

- SMM Device Calibration and Test Messages: Those messages are designed for the calibration and testing of sensors (mainly touch) in “System Master” devices. (No official specification yet.)
- SMM Test Server Touch Messages: Those messages are designed to test touch events on the SMM (No official specification yet. Messages described here: https://wiki.bsh-sdd.com/x/x5BOWw)

- Drives Control Diagnostic Messages: Those messages are designed for the diagnostics of motor powered drives.
- Common Bluetooth Module Messages: Those are generic messages used in all Bluetooth modules.
- Generic Appliance Communication Messages: Those messages are designed for a unified communication (with integrated error handling) on “System Master” devices.

5.2.13 D-Bus-2 Update Service Messages

This set of messages has been introduced since D-Bus-2.2 in order to perform software updates on a given node. It can be activated by a macro “DBUS2_UPDATE” or “DBUS2_UPDATE_HSI”. While the first of the named macros does not activate the HSIProtocolRequest and HSIProtocolResponse, the other does.

Generally, the steps needed to prepare an update process are defined by following scheme:

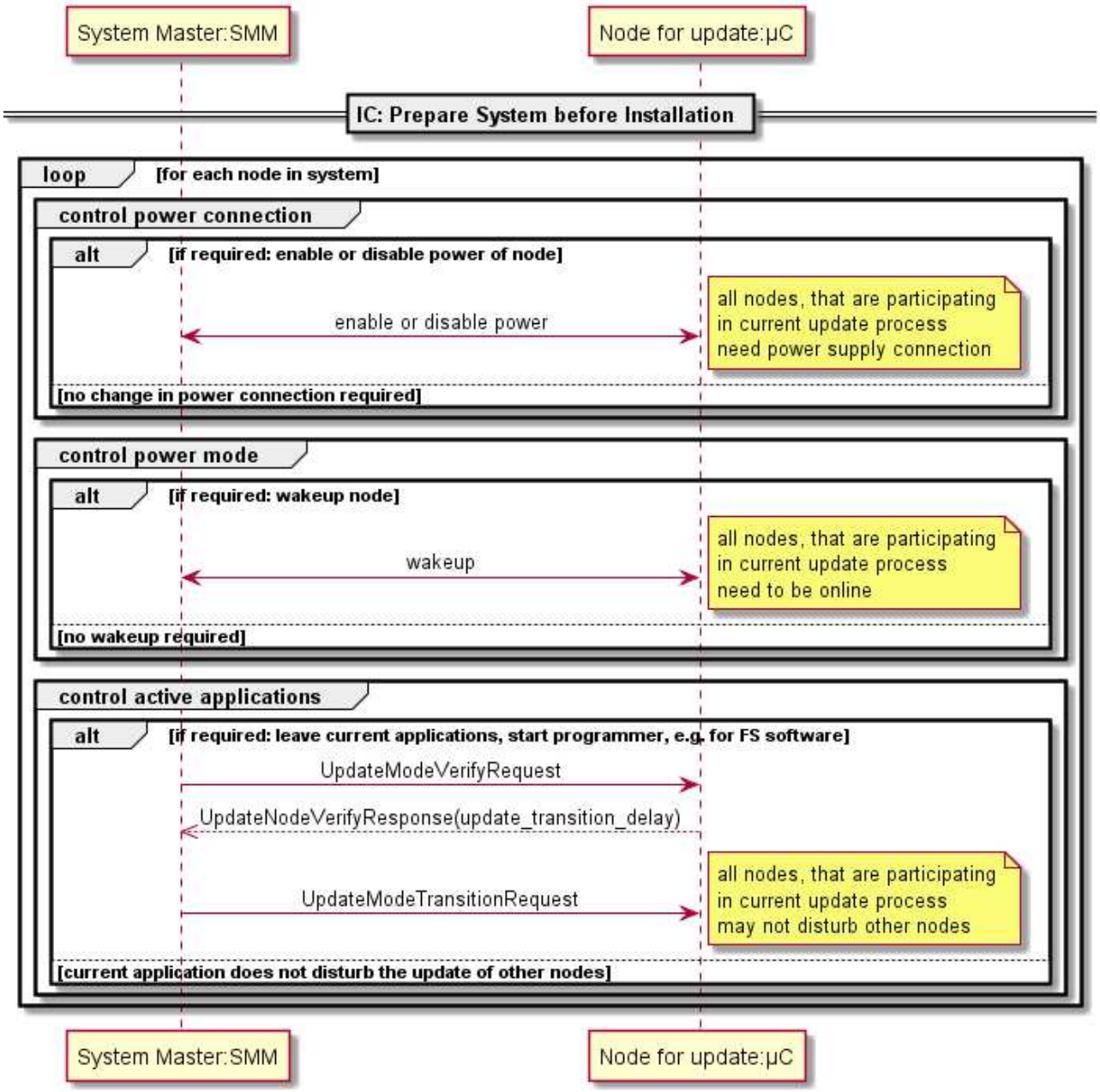


Figure 10: Preparation of Update process.
It should be noted, that it is project specific, which node should be updateable. Therefore, it is a project specific decision, e.g., which node should be powered off during an update.

Once the update has been prepared, in order to update a system of several nodes, the following scheme should be used:

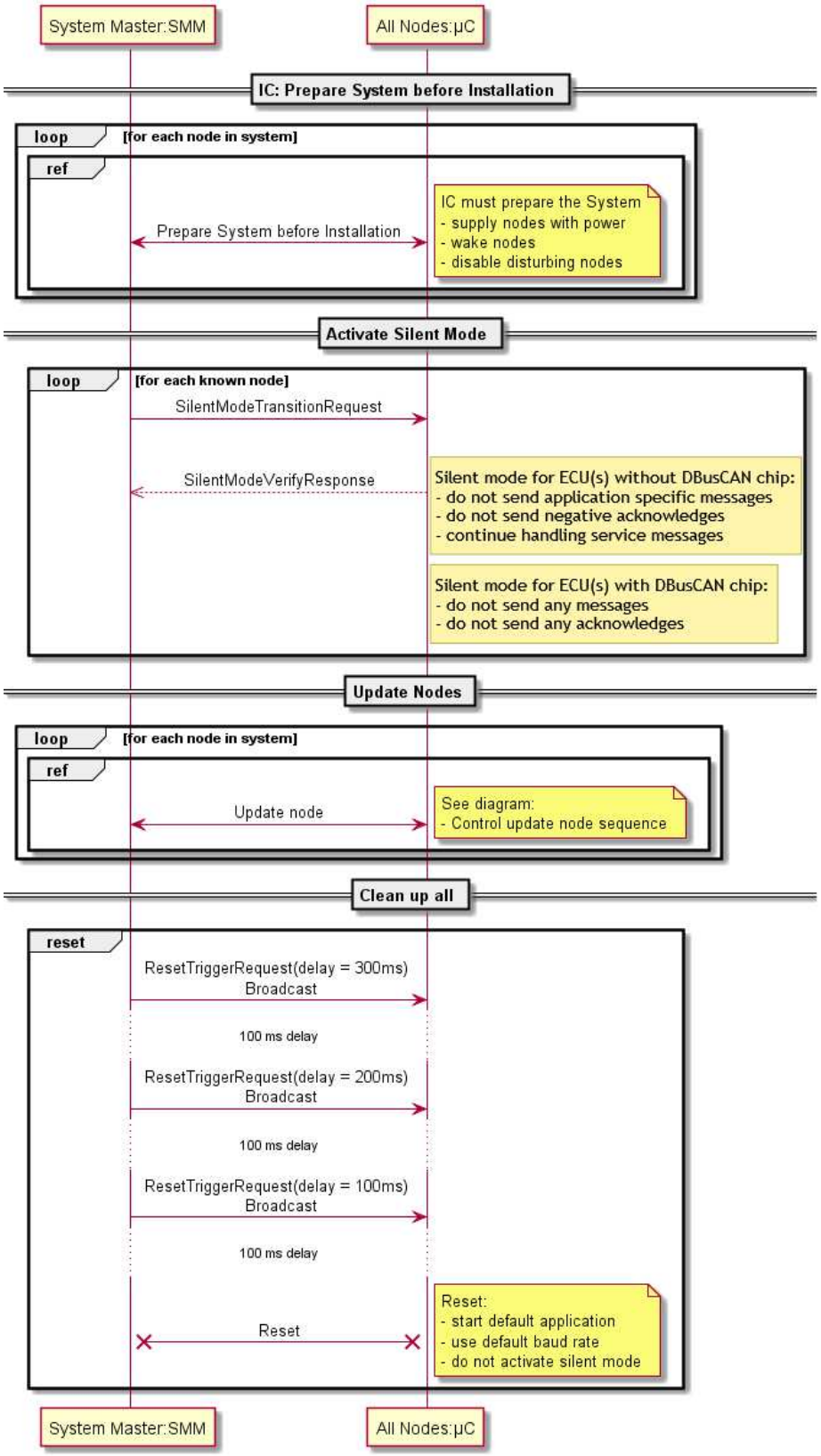


Figure 11: Overview of control sequence in update process.

NOTE concerning SiletModeTransitionRequest: All updatable nodes have to be known, so they all get the directed request. Non-updatable nodes might not be known. However, unkown nodes will receive at least the broadcast request.

Inside the logic of Figure 11, the update of one single node can be described thus:

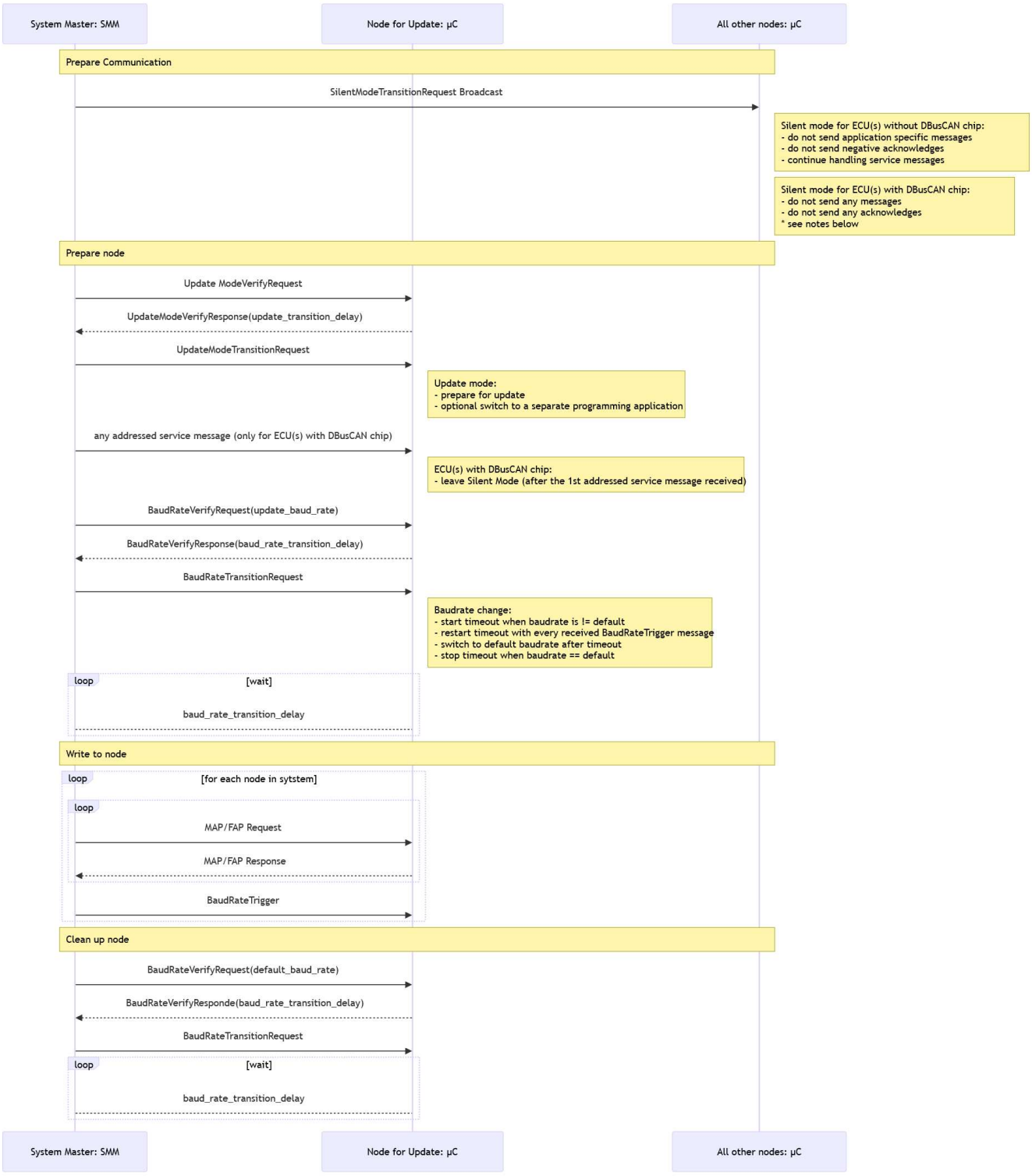


Figure 12: Control of update node sequence

For Figure 11 and for Figure 12, the following should be noted:

- Every request, for which a response has been expected, but not received, must be repeated up to 3 times. In particular, for the “Update Mode Verify Request” (chapter 5.2.13.3) and the “Baud Rate Verify Request”

(chapter 5.2.13.6), it should be noted, that if no response is received after three repetitions, the update mode/baud rate is actually not supported by the addressed node.

- In any error state during the update, for which no other behavior has been explicitly defined, we must first wait for the baud rate reset time (chapter 5.2.13.8 & 5.2.13.9) to pass, plus another 5 seconds. After that, all nodes must be explicitly set to the Silent Mode. Finally, Reset Trigger Requests must be issued to all nodes. That way, the process can be started from the beginning.
- After directed messages of the type "Silent Mode Transition Request" to all nodes, a broadcast should always be sent, just in case any of the addressed nodes has performed a reset or otherwise left the Silent Mode.
- If a MAP/FAP message cannot be processed correctly, when sent with high baudrate (including 3 repetitions), the baud rate must be set to project default and the message must be repeated. After the problematic message has been processed correctly, the remaining part of the update must be continued with high baud rate.
- MAP/FAP requests and responses are embedded into HSI frames, which are transported by the "HSI Protocol Request" and the "HSI Protocol Response".
- HSI Frames are only availeable in Update Mode.
- Only one master at a time is allowed to perform updates or to send Update Service Messages.
- ***As of now (04.09.2019) the master for the update actually is implemented in such a way, that ALL nodes, which can be updated, are put into Update Mode, before the update of the respective nodes is started. It was found, that this procedure is within this specification and does not have to be changed.***
- ECU(s) with DBusCAN chip in the *Update Mode* leave(s) the Silent Mode by receiving any ***addressed*** service message automatically. Because of the technical conditions of the DBusCAN chip the first ***addressed*** service message will run into an ACK timeout.

The following descriptions shall give more details about the correct usage of Dbus2 Update Service Messages, and also about the correct behavior of slaves, that support these messages.

5.2.13.1 Silent Mode Transition Request

The application is put into Silent Mode.

ECU(s) without DBusCAN chip:
Application will send responses only to service messages and acknowledge only correct dbus frames, which means that only ack ok is sent.

ECU(s) with DBusCAN chip:
Application shall not create any acknowledge or response to incoming messages.

This state must be reached no later, than 100ms after the request has been sent.

For a directed communication, the request looks thus:

Message Identifier	**Msg Parameter1** **Source Address**
F501h MSG_SILENT_MODE_TRANSITION_REQUEST	Node / SubSys

Length: 3 Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;         //message identifier
  uchar ucSenderAddress;              //sender address
} TsilentModeTransitionRequest;
```

In this case, the addressed node must send a "Silent Mode Transition Response" (chapter 5.2.13.2).

For a broadcast, the request looks thus:

Message Identifier
F502h MSG_SILENT_MODE_TRANSITION_REQUEST_BROADCAST

Length: 2 Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;         //message identifier
} TsilentModeTransitionRequest;
```

In that case, no response is expected.

When an application enteres the Silent Mode, the function "`DBPL_vSilentModeHasBeenEntered`" is called to inform the application.
ATTENTION: For safety-relevant applications, please look at the coexisting documents concerning functional safety (chapter 1.3). This is important, because in Silent Mode, your application will be frequently receiving interrupts, while other participants on the dbus are being programmed.

5.2.13.2 Silent Mode Transition Response

Confirmation, that the silent mode has been entered. It is issued in response to requests, which are not broadcasts.

Message Identifier
F503h MSG_SILENT_MODE_TRANSITION_RESPONSE

Length: 2 Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;         //message identifier
} TsilentModeTransitionResponse;
```

5.2.13.3 Update Mode Verify Request

By this message, the addressed node is asked, whether an update is possible.

Message Identifier	Msg Parameter1 Source Address
F504h MSG_UPDATE_MODE_VERIFY_REQUEST	Node / SubSys

Length: 3 Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;         //message identifier
  uchar ucSenderAddress;              //sender address
} TupdateModeVerifyRequest;
```

If an update is possible, the addressed node must send an "Update Mode Verify Response" (chapter 5.2.13.4), otherwise not. The receiving timeout for the response is 100ms.

5.2.13.4 Update Mode Verify Response

This message gives the information, in what time it is possible to switch to the update mode. It is a response to the "Update Mode Verify Request".

Message Identifier	Msg Parameter1 Transition Delay
F505h MSG_UPDATE_MODE_VERIFY_RESPONSE	In units of 10ms

Length: 4 Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;         //message identifier
  uint16 uTransitionDelay;              //transition delay
} TupdateModeVerifyResponse;
```

It is sent, if the update is possible. The Transition Delay contains the time, needed by the hardware for the switch into update mode, as well as some additional time for receiving and processing the "Update Mode Transition Request" (chapter 5.2.13.5).

5.2.13.5 Update Mode Transition Request

This command makes the addressed node to switch into update mode, if available.

Message Identifier
F506h MSG_UPDATE_MODE_TRANSITION_REQUEST

Length: 2 Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;         //message identifier
} TupdateModeTransitionRequest;
```

The time needed for transition into update mode can be obtained from the "Update Mode Verify Response". It should be counted, starting at the moment, at which the request has been sent.

5.2.13.6 Baud Rate Verify Request

This message is to request, whether the given baud rate is supported by the addressed node. Before performing a baud rate transition (chapter 5.2.13.8), it has to be requested, whether a baud rate is supported.

Message Identifier	Msg Parameter1 Sender Address	Msg Parameter2 Requested Baud Rate
F507h MSG_BAUDRATE_VERIFY_REQUEST	Node / SubSys	In units of 100 Bits/s

Length: 5 Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;         //message identifier
  uchar ucSenderAddress;              //sender address
  uint16 uRequestedBaudRate;            //baud rate
} TbaudRateVerifyRequest;
```

As of now, the following baud rates can be supported[10]:

```
typedef enum {
  DBPL_Baud9600=96U,
  DBPL_Baud19200=192U,
  DBPL_Baud38400=384U,
  DBPL_Baud57600=576U,
  DBPL_Baud115200=1152U,
  DBPL_Baud125000=1250U,
  DBPL_Baud230400=2304U,
  DBPL_Baud250000=2500U,
  DBPL_Baud500000=5000U,
  DBPL_Baud1000000=10000U
}T_DBPL_BaudRate;
```

In order to make an update work properly, the update baud rate should not be a multiply of the standard baud rate. This precaution makes it more difficult for other nodes, which are not being updated, to see a valid dbus message in their lower baud rate and disturb the update by an ack_ok. Furthermore, even if this should happen, if the update baud rate is not exactly a multiply of the standard, the slower observer will see different bytes on a second attempt, thus not disturbing the update anymore.

If the requested baud rate is supported, the addressed node must send a "Baud Rate Verify Response" (chapter 5.2.13.7), otherwise not. The receiving timeout for the response is 100ms.

[10] Baudrates 500k and 1M available only for DBusCAN chip. However, DBusCAN chip does not support all baudrates listed. See DBusCAN chip specification in chapter 1.3 for more details.

5.2.13.7 Baud Rate Verify Response

This message is a response to the "Baud Rate Verify Request". It is issued, if the requested baud rate is supported.

Message Identifier	Msg Parameter1 Transition Delay
F508h MSG_BAUDRATE_VERIFY_RESPONSE	In units of 10ms

Length: 4 Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;         //message identifier
  uint16 uTransitionDelay;              //transition delay
} TbaudRateVerifyResponse;
```

The "Transition Delay "contains both the time, needed by the hardware to switch the baud rate, and also some additional time for receiving and processing the "Baud Rate Transition Request" (chapter 5.2.13.8).

5.2.13.8 Baud Rate Transition Request

If the baud rate requested previously by the "Baud Rate Verify Request" is supported, this command causes the node to switch to that baud rate, and otherwise there is no effect.

Furthermore, a timer is started, to switch the baud rate back to default after 35 seconds.
While it is running, the baud rate timer can be adjusted by the "Baud Rate Trigger Request" command (chapter 5.2.13.9).

Message Identifier
F509h MSG_BAUDRATE_TRANSITION_REQUEST

Length: 2 Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;         //message identifier
} TbaudRateTransitionRequest;
```

The time needed by the node to execute the request can be obtained from the "Baud Rate Verify Response" and should be counted, starting at the moment, at which the request has been sent.

Typically, as long as the non-standard baud rate is needed, the "Baud Rate Trigger Request" should be issued every 10 seconds to set the timer to 35 seconds.

Furthermore, if the application performs a reset, while the non-standard baud rate is set, this means, that afterwards, the standard baud rate will be set.

5.2.13.9 Baud Rate Trigger Request

This command sets the baud rate timer in the addressed node to the specified value, in order to keep the previously set, non default baud rate for a longer time.

If the default baud rate is set at the time, that message has no effect.

Message Identifier	Msg Parameter1 Time
F50Ah MSG_BAUDRATE_TRIGGER_REQUEST	In units of 10ms

Length: 4 Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;         //message identifier
  uint16 uTime;                         //time
} TbaudRateTriggerRequest;
```

Typically, as long as the non-standard baud rate is needed, the "Baud Rate Trigger Request" should be issued every 10 seconds to set the timer to 35 seconds.

5.2.13.10 Reset Trigger Request

That command causes the addressed node to perform a reset and leave the update mode after the specified time.

Message Identifier	Msg Parameter1 Time Delay
F50Bh MSG_RESET_TRIGGER_REQUEST	Units of 10ms

Length: 3 Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;         //message identifier
  uchar uTriggerDelay;                //delay time
} TresetTriggerRequest;
```

Furthermore, this command will cause the node to leave the Silent Mode and switch to the standard baud rate. Typically, this message is to be send as a broadcast after the entire device has been updated, in order to resume normal operation. (For details, see description at beginning of chapter 5.2.13.)

5.2.13.11 HSI Protocol Request

That command transports HSI frames from a master to a slave device.

Message Identifier	Msg Parameter1 Sender Address
F50Ch MSG_HSI_PROTOCOL_REQUEST	Node / SubSys

Length: 3-251 Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;         //message identifier
  uchar ucSenderAddress;              //sender address
  uchar aucData[MAX_LENGTH];          //HSI Frame
} ThsiProtocolRequest;
```

At the beginning of the processing of this request, the baud rate timer is to be stopped. It must continue running down, the moment, at which the sending of the “HSI Protocol Response” is triggered. This way, we can be sure, that the baud rate is not falsely reset to standard, even if the processing of the “HSI Protocol Request” takes longer, than the maximum time that can be set in the baud rate timer or if we do not know the time needed for the processing.

Consequently, the master in the update process has to add the time between sending the “HSI Protocol Request” and receiving the “HSI Protocol Response” to the time, after which the baud rate is expected to switch back to standard.

5.2.13.12 HSI Protocol Response

That command transports HSI frames from a slave to a master device.

Message Identifier
F50Dh MSG_HSI_PROTOCOL_RESPONSE

Length: 2-251 Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;         //message identifier
  uchar aucData[MAX_LENGTH];          //HSI Frame
} ThsiProtocolResponse;
```

5.2.13.13 Return from Silent Mode Request

This request may be used to make a node leave the Silent Mode, only if booting takes a lot of time (at least seconds), and for some reason, it cannot be afforded to wait that long for a complete reset to be performed. Specifically, this might be the case in production.

Message Identifier	Msg Parameter1 Source Address
F50Eh MSG_RETURN_FROM_SILENT_MODE_ REQUEST	Node / SubSys

Length: 3Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg; //message identifier
  uchar ucSenderAddress;      //sender address

} TreturnFromSilentModeRequest;
```

In case of success a “Return from Silent Mode Response” (chapter 5.2.13.14) must be issued by the addressed node. The Silent Mode must be left not later, than 100ms after the request has been issued.

5.2.13.14 Return from Silent Mode Response

This message confirms, that the addressed node has successfully returned from Silent Mode.

Message Identifier
F50Fh MSG_RETURN_FROM_SILENT_MODE_ RESPONSE

Length: 2Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;         //message identifier
} TreturnFromSilentModeResponse;
```

5.2.13.15 ECU Config Read Request

This message can be used in an update process to get information about the hardware and software of a node.

Message Identifier	Msg Parameter1 Sender Address	Msg Parameter2 Number of Identification Object
F510h MSG_ECU_CONFIG_READ_REQUEST	Node / SubSys	

Length: 4Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg; //message identifier
  uchar ucSenderAddress;      //sender address
  uchar ucIdObjectNumber;     //numbder of identification object
} TecuConfigReadRequest;
```

For any software, the addressed node must have one Identification Object, which contains the identification and version of both software and hardware. The first “ECU Config Read Request” must always be sent for “Identification Object” number one. (It must be noted that zero is prohibited as a number of Identification Objects.) The “ECU Config Read Response” (chapter 5.2.13.16) will contain both the requested object and the overall count of Identifictaion Objects. If the count is bigger than one, the remaining objects must also be collected, using succeeding numbers to identify the requested objects.

Each Identification Object contains information about a software and a hardware on which it is running. This is important, because in some cases, it might be dependant on the version of the hardware, which software can be programmed during an update. If there are, e.g., four different types of software on one ECU, there will be four Identification Objects, each containing exactly the same data about the hardware, but differing data about the software.

The receiving timeout for the “ECU Config Read Response” is one second.

5.2.13.16 ECU Config Read Response

This message returns the Identification Object requested by the “ECU Config Read Request”, as well as the overall count of objects and a status indicating, whether the attempt to access the object has been successful.

Message Identifier	Msg Param 1	Msg Param2	Msg Param3	Msg Param4	Msg Param5
	Sender Address	Status	No. of Id Object	Count of Objects	Identification Object
F511h MSG_ECU_CONFIG_READ_RESPONSE	Node / Subsys	0-OK 1-Update Mode Active 32-Error 33-Unsupported			

Length: 42Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg //message identifier
  uchar ucSenderAddress;     //Address, with which sender has been addressed in last received message
  uchar status;
  uchar ucIdObjectNumber;
  uchar ucObjectCount;
  struct identificationObject{
    uchar hwID[8];           //hardware Identifier, 64-Bit value, Big Endian
    uint16 hwVersionMajor;
    uint16 hwVersionMinor;
    uint16 hwVersionRevision;
    uint32 hwVersionBuild;
    uchar swID[8];           //software Identifier, 64-Bit value, Big Endian
    uint16 swVersionMajor;
    uint16 swVersionMinor;
    uint16 swVersionRevision;
    uin32 swVersionBuild;
  };
} TecuConfigReadResponse;
```

Any Ecu, that supports the update process for software, must return a set of valid values in the "ECU Config Read Response".

An Ecu, which does not support the update of software, might as well return dummy objects, in which the "Identification Object" number is the only parameter with a valid value, while the other parameters have zero as a value. In such a case, the decision between returning either proper or dummy values is up to the leader of the given project.

5.2.13.17 ECU Software Submodule Read Request

This message is to be used to query the version of a generic library inside a concrete software on an ECU. It is designed mainly for Customer Service and other instances that might want to access and examine a device locally.

Message Identifier	Msg Parameter1	Msg Parameter 2	Msg Parameter3
	Sender Address	Software Identifier	Number of Identification Object
F512h MSG_ECU_SOFTWARE_SUBMODULE_READ_REQUEST	Node / SubSys	Same as obtained in "Ecu Config Read Response"	

Length: 12Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg; //message identifier
  uchar ucSenderAddress;      //sender address
  uchar swID[8];              //software Identifier, 64-Bit value, Big Endian
  uchar ucIdObjectNumber;     //numbder of identification object
} TecuFirmwareSubmoduleReadRequest;
```

The user has to send the first request for Identification Object number one. (It must be noted that zero is prohibited as a number of Identification Objects.) The "Ecu Software Submodule Read Response" will contain the requested object and the overall number of identification objects. The queries must be continued for consecutive identification object numbers until the overall number is reached. If the overall number is zero, there is no information about submodules available.

Each Identification object contains information about the software as such as well as the submodule.

The receiving timeout for the "ECU Software Submodule Read Response" is one second.

5.2.13.18 ECU Software Submodule Read Response

This message returns the Identification Object requested by the "ECU Software Submodule Read Request", as well as the overall count of objects and a status indicating, whether the attempt to access the object has been successful.

Message Identifier	Msg Param 1	Msg Param2	Msg Param3	Msg Param4	Msg Param5
	Sender Address	Status	No. of Id Object	Count of Objects	Identification Object
F513h MSG_ECU_SOFTWARE_SUBMODULE_RESPONSE	Node / Subsys	0-OK 1-Update Mode Active 32-Error 33-Unsupported			

Length: 42Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg //message identifier
  uchar ucSenderAddress;     //Address, with which sender has been addressed in last received message
  uchar status;
  uchar ucIdObjectNumber;
  uchar ucObjectCount;
  struct identificationObject{
    uchar swID[8];           //software Identifier, 64-Bit value, Big Endian
    uint16 swVersionMajor;
    uint16 swVersionMinor;
    uint16 swVersionRevision;
    uint32 swVersionBuild;
    uchar submoduleID[8];    //submodule Identifier, 64-Bit value, Big Endian
    uint16 submoduleVersionMajor;
    uint16 submoduleVersionMinor;
    uint16 submoduleVersionRevision;
    uin32 submoduleVersionBuild;
  };
} TecuFirmwareSubmoduleReadResponse;
```

5.2.14 Production Service Messages

Those messages are an addition to the "Update Service Messages" in order to program and read production data and are mostly available for those nodes that support "Update Mode". The messages for both reading and writing the "Test State" are also available for those nodes without "Update Mode". If messages for reading and writing "Appliance Data" are needed, they may be implemented (together with the other writing messages) by all devices, no matter, if they support "Update Mode" or not. Apart from the "Test State", production data must be programmed (in "Update Mode", if available) according to the same scheme that has been defined in the previous section about "Update Service Messages". (In such a case, the "HSI Protocol Request" is replaced by the command for writing the given production data.) All the reading messages are to be available outside of "Update Mode".

The response timeout for all "Production Service Messages" is one second.

5.2.14.1 Write Ecu Config Hw Request

This message is to write the hardware part of the Ecu Config. It is to be answered by "Write Ecu Config Hw Response" (chapter 5.2.14.2). The hardware part of the Ecu Config can be written exactly once.

Message Identifier	Msg Param 1	Msg Param2
	HW ID	HW Version
F312h MSG_WRITE_ECU_CONFIG_HW_REQUEST	uint64	3*uint16+uint32

Length: 20Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg //message identifier
  uchar hwID[8];             //hardware Identifier, 64-Bit value, Big Endian
  uint16 hwVersionMajor;
  uint16 hwVersionMinor;
  uint16 hwVersionRevision;
  uint32 hwVersionBuild;
}TwriteEcuConfigHwRequest;
```

All parameters are to be hexadecimal.

5.2.14.2 Write Ecu Config Hw Response

This message returns the status of the attempt to write the hardware part of the Ecu Config. It is always to be sent to node address 0xC0.

Message Identifier	Msg Param 1 Status
F313h MSG_WRITE_ECU_CONFIG_HW_RESPONSE	**enum** Status{ *OK* = 0xA0U, *Param1Error* = 0xB1U, *Param2Error* = 0xB2U, *Param3Error* = 0xB3U, *LengthError* = 0xC0U, *UnknownMsgError* = 0xC1U, *FunctionError* = 0xD0U };

Length: 3Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg //message identifier
  uchar status;
}TwriteEcuConfigHwResponse;
```

5.2.14.3 Write Tracing Id Request

This message is for writing the Tracing ID of a PCB. It is answered by the "Write Tracing Id Response" (chapter 5.2.14.4). The Tracing ID can be written exactly once.

Message Identifier	Msg Param 1 Material Number	Msg Param2 SupplierID	Msg Param3 Counter
F314h MSG_WRITE_TRACING_ID_REQUEST	10 Bytes	10 Bytes	9 Bytes

Length: 31Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg; //message identifier
  uchar materialNumber[10];
  uchar supplierId[10];
  uchar counter[9];
}TwriteTracingIdRequest;
```

All parameters are BCD coded. Each character from the Tracing ID string is to be one byte in the message.

5.2.14.4 Write Tracing Id Response

This message returns the status of the attempt to write the Tracing ID. It is always to be sent to node address 0xC0.

Message Identifier	Msg Param 1 Status
F315h MSG_WRITE_ TRACING_ID _RESPONSE	**enum** Status{ *OK* = 0xA0U, *Param1Error* = 0xB1U, *Param2Error* = 0xB2U, *Param3Error* = 0xB3U, *LengthError* = 0xC0U, *UnknownMsgError* = 0xC1U, *FunctionError* = 0xD0U };

Length: 3Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg //message identifier
  uchar status;
}TwriteTracingIdResponse;
```

5.2.14.5 Write Production Time Request

This message is for writing the production time of a PCB. It is answered by the "Production Time Response" (chapter 5.2.14.6). The production time can be written exactly once.

Message Identifier	Msg Param 1 Production Date	Msg Param2 Production Clock
F316h MSG_WRITE_PRODUCTION_TIME_REQUEST	6 Bytes DD DD MM MM YY YY BCD coded (one digit one byte, e.g.:) 00 07 00 09 01 09	4 Bytes HH HH MM MM BCD coded (one digit one byte, e.g.:) 01 04 03 00

Length: 12Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg //message identifier
  uchar productionDate[6];
  uchar productionClock[4];
}TwriteProductionTimeRequest;
```

All parameters are BCD coded.

5.2.14.6 Production Time Response

This message returns the status of the attempt to write/read the production time and the time as such. It is always to be sent to node address 0xC0.

Message Identifier	Msg Param 1 Status	Msg Param 2 Production Date	Msg Param3 Production Clock
F317h MSG_ PRODUCTION_TIME _RESPONSE	**enum** Status{ *OK* = 0xA0U, *Param1Error* = 0xB1U, *Param2Error* = 0xB2U, *Param3Error* = 0xB3U, *LengthError* = 0xC0U, *UnknownMsgError* = 0xC1U, *FunctionError* = 0xD0U };	6 Bytes DD DD MM MM YY YY BCD coded (one digit one byte, e.g.:) 00 07 00 09 01 09	4 Bytes HH HH MM MM BCD coded (one digit one byte, e.g.:) 01 04 03 00

Length: 13Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg //message identifier
  uchar status;
  uchar productionDate[6];
  uchar productionClock[4];
}TproductionTimeResponse;
```

The parameters 2 and 3 are BCD coded.

5.2.14.7 Read Production Time Request

This message is for reading the production time of a PCB. It is answered by the "Production Time Response" (chapter 5.2.14.6).

Message Identifier
F318h MSG_READ_PRODUCTION_TIME_REQUEST

Length: 2Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg //message identifier
}TreadProductionTimeRequest;
```

5.2.14.8 Write Teststate Request

This message is for writing the Teststate of a PCB in production. It is answered by the "Teststate Repaircount Response" (chapter 5.2.14.9).

Message Identifier	Msg Param 1 Teststate
F319h MSG_WRITE_TESTSTATE_REQUEST	**enum** Teststate{ *Failed* = 0x00U, *OK = 0x01* };

Length: 3Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg //message identifier
  uchar testState;
}TwriteTestStateRequest;
```

The Teststate is always to be stored in an external flash (EEPROM or emulated EEPROM). It is advisable, to increment the Repaircount, in each step after which the Teststate is written. That way, it is easy to keep track of the number of steps taken in production.

5.2.14.9 Teststate Repaircount Response

This message returns the status, the Teststate and the Repaircount of a PCB, both after the attempt to write the Teststate and after the attempt to read Teststate+Repaircount. It is always to be sent to node address 0xC0.

Message Identifier	Msg Param 1 Status	Msg Param 2 Teststate	Msg Param 3 Repaircount
F31Ah MSG_TESTSTATE_REPAIRCOUNT_RESPONSE	**enum** Status{ *OK* = 0xA0U, *Param1Error* = 0xB1U, *Param2Error* = 0xB2U, *Param3Error* = 0xB3U, *LengthError* = 0xC0U, *UnknownMsgError* = 0xC1U, *FunctionError* = 0xD0U };	**enum** Teststate{ *Failed* = 0x00U, *OK = 0x01* };	Valid values are from 0x00 to 0x07

Length: 5Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg //message identifier
  uchar status;
  uchar testState;
  uchar repairCount;
}TtestStateRepairCountResponse;
```

If an invalid value of the Repaircount is returned, the PCB must be disposed of.

5.2.14.10 Read Teststate Repaircount Request

This message is for reading the Teststate and Repaircount of a PCB in production. It is answered by the "Teststate Repaircount Response" (chapter 5.2.14.9).

Message Identifier
F31Bh MSG_READ_TESTSTATE_REPAIRCOUNT_REQUEST

Length: 2Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg //message identifier
}TreadTestStateRepairCountRequest;
```

5.2.14.11 Write Appliance Data Request

This message is for writing Appliance Data of a PCB/controller. It is answered by the "Write Appliance Data Response" (chapter 5.2.14.12). All data can be written exactly once.

Message Identifier	Msg Param 1 Sender	Msg Param 2 Parameter ID	Msg Param 3 Data
F31Ch MSG_WRITE_APPLIANCE_DATA_REQUEST	Node Address		0 – 64 Bytes

Length: 4 - 68Bytes

Type:

```
typedef struct {
  TbusMessageIdentifier tMsg //message identifier
  uchar Sender;
  uchar ParameterId;
  uchar[0-64] Data;
}TwriteApplianceDataRequest;
```

Following table describes which parameter is sent and which size it has, depending on the "Parameter ID":

Parameter Id	Type	Size (bytes)	Description
0	deviceType	1	Uint8 [32;191] See Table 15b
1	deviceTypeString	1-32	String (UTF-8)
2	brand	1-32	String (UTF-8)
3	BMrk	3	String (UTF-8)
4	vib	1-32	String (UTF-8)
5	kiAlphanumeric	2	String (ASCII)
6	fdNumber	4	String (ASCII)
7	serialNumber	0-64	String (UTF-8)
8	manufacturingTimestamp	12	UTF-8: YY YY YY YY MM MM DD DD hh hh mm mm
9	countrySettings	1	Uint8: 0-Worldwide minimal requirements (WW); 1-USA and North America (US); 2-Europe, Asia Pacific without China (EU/AP); 3-China (CN)
10	BLE TX Power	2	Int16, Range -30…200, Default: 80. Defines Tx power level for RF/BLE communication

Table 15a: Parameter Ids, all string parameters are sent without /0 termination (the stated size is without termination)

Following table presents device types:

Category	Device Type	value [uint8_t]
n/a	All (allowed device types, except reserved)	0x00
	Reserved	0x01
	Application (A general kind of communication partner of the home appliance [e.g. App, Backend, Energy Manager…])	0x02
	Reserved	0x03-0x0F
	AllHomeAppliances (anything between 0x20 and 0xBF, except all Reserved in-between)	0x10
	Reserved	0x11-0x1F
DishCare	Dishwasher	0x20
	Reserved	0x21-0x3F

Cooking	Oven	0x40
	Hob	0x41
	Hood	0x42
	RangeCooker	0x43
	Microwave	0x44
	Steamer	0x45
	VentingCooktop	0x46
	Waterbase	0x47
	WarmingDrawer	0x48
	Reserved	0x49-0x5F
Refrigeration	Refrigerator	0x60
	Freezer	0x61
	FridgeFreezer	0x62
	WineCooler	0x63
	Reserved	0x64-0x7F
LaundryCare	Washer	0x80
	Dryer	0x81
	WasherDryer	0x82
	Reserved	0x83-0x9F
ConsumerProducts	CoffeeMaker	0xA0
	WaterHeater	0xA1
	HotWaterHeatPump	0xA2
	ConsumerProduct **(not used and deprecated)**	0xA3
	CleaningRobot	0xA4
	CookProcessor	0xA5
	KitchenRobot	0xA6
	Blender	0xA7
	SmartGrow	0xA8
	KitchenMachine	0xA9
	Reserved	0xA0
	VacuumCleanerHandstick	0xAA
	Reserved	0xAB-0xBF
Generic	Appliance (Generic device type. Provides limited connectivity access [pairing, remote diagnostics, firmware update] to correct an incomplete appliance configuration and/or SW to a proper device type.)	0xC0
	DataLogger (SMM based external logging tool for sensors and Dbus2)	0xC1
	Reserved	0xC2-0xFF

Table 15b: Device Types

5.2.14.12 Write Appliance Data Response

Message Identifier	**Msg Param 1** **Parameter ID**	**Msg Param 2** **Status**
F31Dh MSG_WRITE_APPLIANCE_DATA_RESPONSE	Table 15a (chapter 5.2.14.11)	**enum** Status{ *OK* = 0xA0U, *LENGTH_ERROR* = 0xC0U, *FUNCTION_ERROR* = 0xD0U, *EMPTY_ITEM_ERROR* = 0xD1U, *UNKNOWN_PARAMTER_ID_ERROR* = 0xE0U };

Length: 4Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg //message identifier
  uchar ParameterId;
  uchar Status;
}TwriteApplianceDataResponse;
```

5.2.14.13 Read Appliance Data Request

This message may be used to read Appliance Data from a participant's memory. It is answered by the "Read Appliance Data Response" (chapter 5.2.14.14).

Message Identifier	Msg Param 1 Sender	Msg Param 2 Parameter ID
F31Eh MSG_READ_APPLIANCE_DATA_REQUEST	Node Address	Table 15a (chapter 5.2.14.11)

Length: 4Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg //message identifier
  uchar Sender;
  uchar ParameterId;
}TreadApplianceDataRequest;
```

5.2.14.14 Read Appliance Data Response

Message Identifier	Msg Param 1 Sender	Msg Param 2 Parameter ID	Msg Param 3 Status	Msg Param 4 Data
F31Fh MSG_READ _APPLIANCE_DATA_RESPONSE	Node Address	Table 15a (chapter 5.2.14.11)	**enum** Status{ *OK* = 0xA0U, *LENGTH_ERROR* = 0xC0U, *FUNCTION_ERROR* = 0xD0U, *EMPTY_ITEM_ERROR* = 0xD1U, *UNKNOWN_PARAMTER_ID_ERROR* = 0xE0U };	0 – 64 Bytes

Length: 5 - 69Bytes
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg //message identifier
  uchar Sender;
  uchar ParameterId;
  uchar Status;
  uchar[0-64] Data;
}TreadApplianceDataResponse;
```

5.2.14.15 Factory Reset

This message is used to reset project specific values written during Factory Test. **The message is to be sent as a broadcast.**
Advice: As the message is sent as broadcast, it must be sent 3 times with a time gap of 100ms in between.

Message Identifier
F326h MSG_FACTORY_RESET

Length: 2Bytes
Type:

```
typedef struct {
   TbusMessageIdentifier tMsg //message identifier
}TfactoryResetMessage;
```

5.3 Presentation Layer Application Programmers Interface (API)

The presentation layer contains an interface to the application where the service messages (also called predefined messages, as they are already predefined in this document) are handled. Service messages are mostly used for debug or update reasons. Access to memory is of high importance. When a memory cell (or array) is accessed from e.g. a pc connected to the bus, a memory cell (or array) is meant to be accessed within a certain range of the memory. This range can be the normal memory of the module (RAM/ROM) or external EERPOM as well as specific section of the RAM. To distinguish the different memory ranges a module number is used (see description of section 5.2.3.5 SetMemoryModule message for a description of this aspect). For systems with primitive needs the memory access may be reduced to one memory module (which per default has the value 0).

Presentation Layer Service	Parameter 1	Parameter 2	Parameter 3 … n	Mandatory	Direction[11]
Read Memory	uchar Module	uint16 Address	uchar DataLength, reference to uchar MessageData	X	PresentationL →memory
Write Memory	uchar Module	uint16 Address	uchar DataLength, reference to uchar MessageData		PresentationL→ memory
Get ID Address	uchar Module			X	PresentationL→ memory
Reset	Uchar Delay				PresentationL→user
Stream Mode Active	BOOL Active				PresentationL→user
Read Object	uchar CommunicationObjectIndex	uchar DataLength	reference to uchar DataBlock		PresentationL
Write Object	uchar CommunicationObjectIndex	uchar DataLength	reference to uchar DataBlock		PresentationL

Table 16: Presentation Layer Service Interface

A reset function is present in the presentation layer to provide reset functions also when network management is not implemented. This function is also used (even when network management is implemented) for executing a reset when starting the bootloader.

A function may be provided, ucIsStreamModeActive, for supplying other software modules the possibility to check whether an update is running (e.g. main will be interested in this information for synchronising tasks – some tasks are not allowed during update of the EEPROM).

For a further explanation of the Read- and WriteObject, see reference [1]: Remote extension in Table 2.

5.4 Example: Remote Control Messages deprecated (new: Technologies, that are not UART protocols, Update Service Messages)

See the document extension (separate document) describing the remote access (or remote control) messages for a complete description of use: Reference [1]: Remote extension in Table 2.

All values are hexadecimal.

Message Identifier 16 Bit	Object Data

[11] The term user refers to the overlying layer, which is using presentation layer, i.e. the application. – or the memory access.

RemoteType (1 Byte)	Object (1 Byte)	
1 Byte: 80h (Read) / 81h (ReadWithArg) / 82h (Write) / 88h (ReadResp/Inf) / …	1 Byte	n Byte

Table17: Remote Control Message

According to the description of a frame in section 4.1, a remote control frame consists of 2 byte before the message (data length and TargetAddress) and two byte CRC (+ acknowledgement) after the message. A message can be described as in Table 17.

Start:

A message to start the dishwasher may look like the following example:

82h 03h
MessageID=8203h(Write I@, Object=03h), no object data.

RemainingTime:

A message from the dishwasher, telling that the remaining time is 5 minutes, may look like the following example:

88h 10h 00h 05h
MessageID=8810h (I@-Inf, Object=RemainingTime), 0 Hours, 5 Minutes.

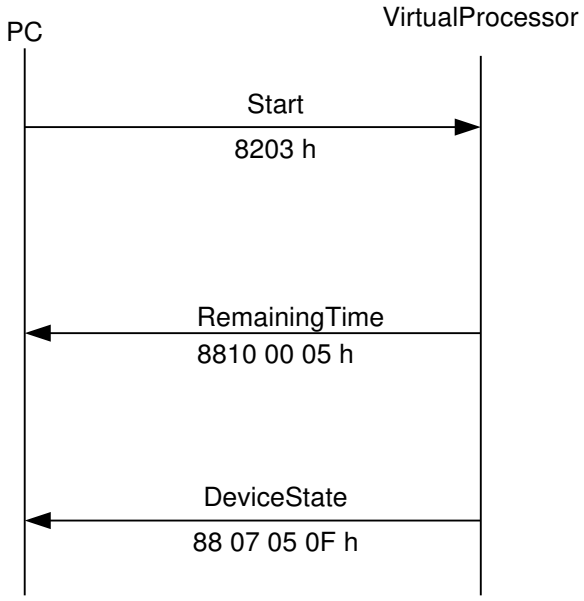
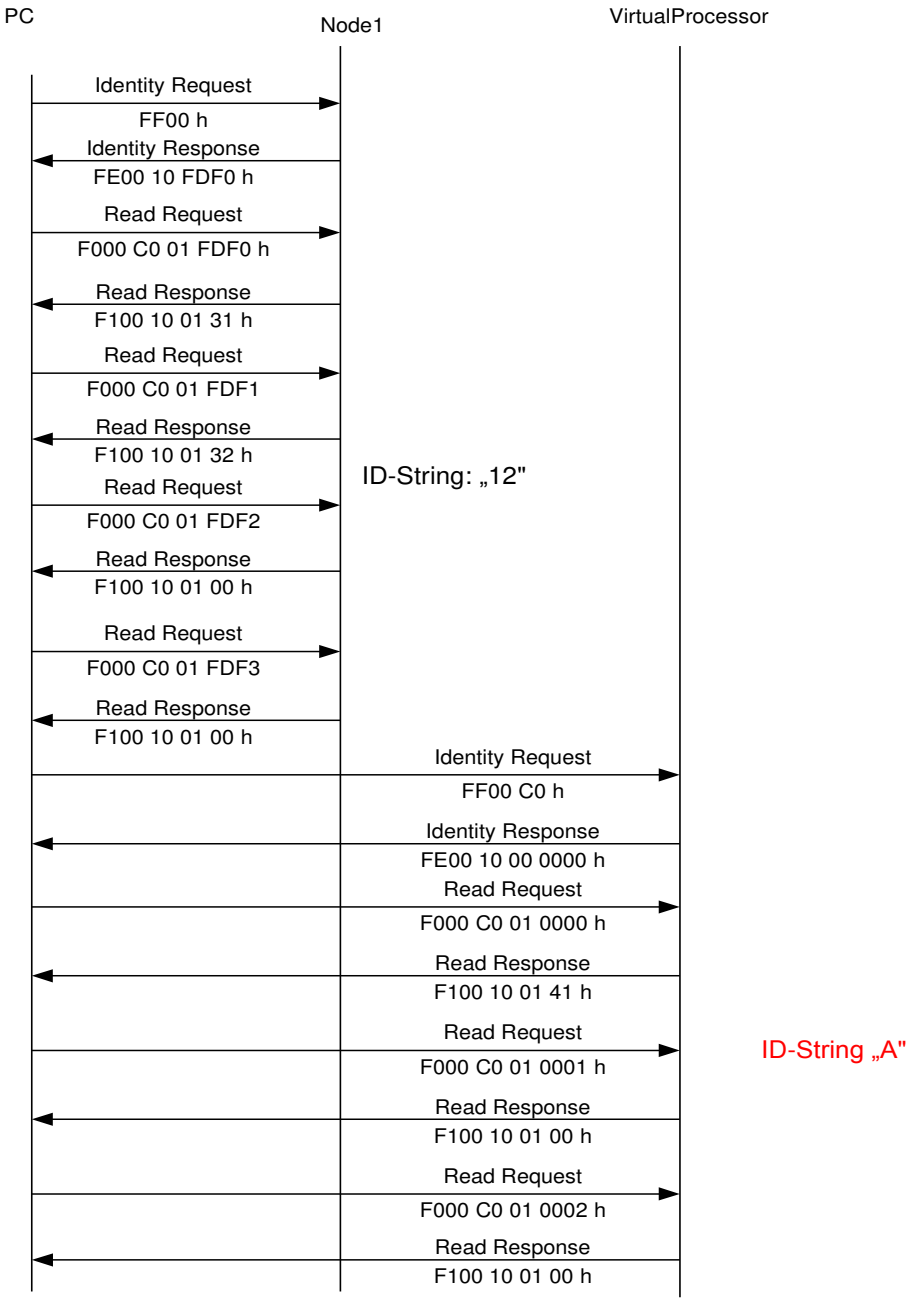


Figure 13: RemoteControl example

DeviceState:
A message from the dishwasher, telling that it is running, and that the remote control is enabled, may look like the following example:
88h 07h 05h 0Fh
MessageID=8807h (I@-Inf, Object=DeviceState), 5=Running, Fh=Remote Enable Active.

5.5 Example: Communication with pc (Service Messages)

Code in hexadecimal values:



Note: If the ID String is not in MemoryModule 0 then the SetMemoryModule command may be called before the IdentityRequest.

6 Application Layer

The application layer (Figure below) includes handling of predefined service messages (described in chapter 5), test messages, remote control messages and the network management messages (see next chapter). The "real" task of the application layer is to distribute application specific messages to and from the application. The distribution is handled via service functions. A service function must be generically defined (equal amount of parameters – also having the same type definitions), since the access follows over tables (see Table 20, Table 21 and Table 22 below).

Remote control messages are handed over to the respective subsystem[12].

Before a message has been handled, further messages may arrive (e.g. through the receive interrupt), as long as the input buffer is not full. If the receive buffer is full, all further messages are discarded (and accordingly acknowledged as busy).

Network management messages are special messages used for e.g. synchronising the network. When network management is implemented, application specific messages may only be exchanged when the network is operational (exception: initialisation data, see chapter 7 for further details).

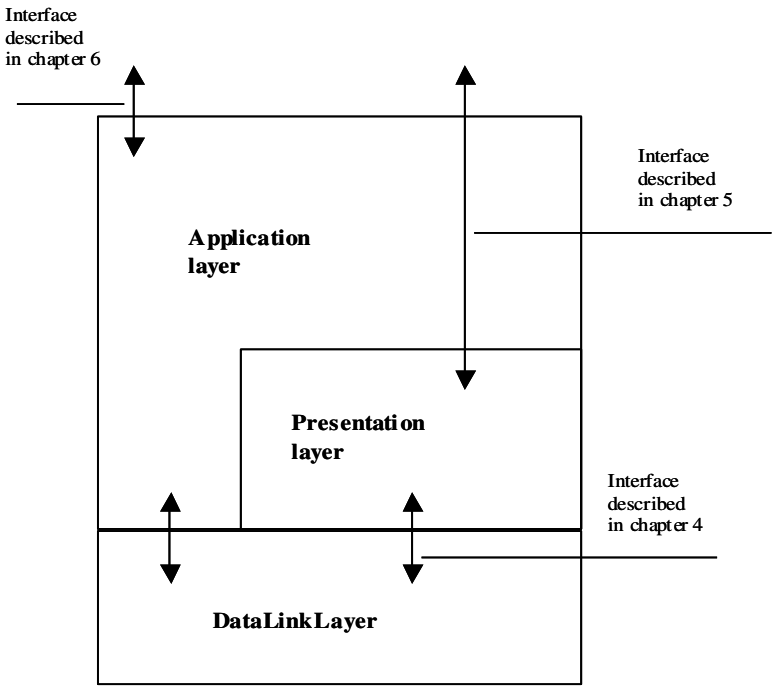


Figure 14: Application Layer

6.1 Message signalling and processing

When a message is received, this is saved and provided to the application via a dedicated service function. The service function is associated with the corresponding message in one of several tables (see Table 20, Table 21 and Table 22 below). The corresponding table is found by decoding the subsystem from the frame. A possibly unknown message will not have an appropriate service function associated, and will be discarded.

Distribution of all incoming and outgoing messages over dedicated, generic, service functions makes the bus being fully exchangeable with another bus using the same generic service functions (assumed that the maximum data length, and similar, is not exceeded). This is why it is important that system messages (application specific messages) are used in this way.

[12] Please note that a remote control message may have to be processed by more than one communication partner. If the subsystem VirtualProcessorRemoteControl does not contain the requested information, the request can be sent to another communication partner via other (application specific) messages. This second node will afterwards answer the SystemInterface (without detouring via the first communication partner again).

When a message has been distributed via a service function, this may be signalised to the application from the service function (the service function could set a variable to a certain value, or call another function). If not signalised from the service function itself, the point of processing finalisation of an incoming message is not explicitly known.

Application Layer Service	Parameter 1	Parameter 2	Mandatory	Direction[13]
Send Message Request	uchar MessageTableIndex	uchar MessageNumber	X	user-> Application Layer

Table 18: Application Layer Service Interface

Application Layer Tables	Mandatory
ReceiveObjectSubsystem-Table	X
TransmitObjectSubsystem-Table	X
ObjectTable	X

Table 19: Application Layer Tables

The service function, which is generic, may look like the following:

```
void SUBSYS_vGetService(uchar ucDataLen, uchar *pucData)
```

The *pucData refers to the exchanged data. Within the service function the data should be accessed by using associated type-definitions, to avoid problems, which may appear by a generic approach, when e.g. the data length of the exchanged data structure is changed. Each message has an associated service function.

BusObject Table
reference to tReceiveObjectSubsystem
reference to tTransmitObjectSubsystem

Table 20: BusObject Table

ReceiveObjectSubsystem Table
TbusIdentifier tBusidentifier
uchar ucDataLength
reference to void ServiceFunction

Table 21: ReceiveObjectSubsystem Table

TransmitObjectSubsystem Table
TbusIdentifier tBusidentifier
uchar ucDataLength
reference to void ServiceFunction
reference to void CompleteFunction

Table 22: TransmitObjectSubsystem Table

Specifically a service function for transmission of a motor speed may look like the following on the transmitting node:

```
void MOTOR_CONTROLLER_vGetTargetSpeed(uchar ucDataLen, uchar *pucData){
  *(MotorTargetValuesStruct *) pucData = &MOTOR_CONTROLLER_MotorTargetValues;
  return;
}
```

- and similarly, on the receiving node:

```
void MOTOR_vSetTargetSpeed(uchar ucDataLen, uchar *pucData){
  MOTOR_MotorTargetValues = *(MotorTargetValuesStruct *) pucData;
  return;
}
```

Transmission of a message follows after a flag has been set (each message has a corresponding flag telling whether it is due to be sent), MessageToSendFlag (boolean). The message associated with the flag is defined in one of several tables. Each subsystem only knows messages defined in the message table associated with its subsystem, i.e. a message belonging to one TransmitObjectSubsystem does not exist in any other TransmitObjectSubsystem.

[13] The term user refers to the overlying layer which is using this layer.

Prioritisation of messages must be done by the calling application – if necessary. The first message to be sent will by default be the first message with a corresponding message to send flag set, according to the order of defined subsystems. Hence if messages from the first and second subsystems are scheduled for transmission; the first message of the first subsystem will be handled first, then the second message, etc.

Typically, one byte will be enough as MessageToSendFlag for 8 messages for each subsystem. If more than 8 messages are needed for one subsystem care must be taken, so that the MessageToSendFlag is accommodated in a proper way (e.g. by using an excess technique (overwriting a variable when linking the project) the one byte MessageToSendFlag-variable can be extended to two or more bytes – description of the excess technique is out of scope of this document).

Each subsystem may send a message to an arbitrary other subsystem within another communication partner. Messages to subsystems within the same communication partner are not supported.

The following function may be called to initiate a transmission:

```
extern void BAL_vTransmitMessage(uchar ucSubsystem, uchar ucMessageNumber)
```

The two parameters used by this function are indexes. The first one refers to the index of the table where the messages are defined (subsystem number), the second refers to the index of the message in the table addressed by the first parameter (message number). The message data is handled in the message service function, which may also tell the application that the message has been processed (the associated data has been gathered, but the message may still not have arrived on the bus):

```
extern void SUBSYS_vSetService(uchar DataLen, uchar *pucData)
```

Please note, that for transmission, the application could be signalised from the service function (telling that data has been gathered for transmission), or from a message finalisation function associated with the sending of the message (message has been sent on the bus).

Signalling that the message has been sent does not reflect whether the addressed subsystem has processed the message.

6.1.1 Application layer transmission

When the application wants to send a message to a subsystem on another communication partner the data must be available via the service function. If a certain value should be transmitted, the application must make sure this value is copied to the service function (otherwise the value might have changed), as the transmission will not follow immediately. If the value should be as recent as possible, the service function must be supplied with the present value (e.g. direct access to a volatile variable).

Assuming the data to transmit is ready, the next step is to signalise the bus message handler by setting the appropriate flag via BUS_vSendMessageRequest (step 1. in Figure). One subsystem does not know any messages from any other subsystems on the same communication partner. Only the messages defined in the current message table are available, hence the first parameter of BUS_vSendMessageRequest refers to the subsystem 1 for Figure. The second parameter refers to the message, which is due to be sent.

The bus message handler will some time later (by low traffic this will typically be within the next bus task call) process this message, as all the flags are checked to see if any message is due to be sent. The bus message handler processes the message transmission by calling the service function registered in the table associated with this message to get the appropriate data (step 2 in Figure). Where to send the message is known, as the bus identifier associated with the message consists of the target address as well as the message identifier.

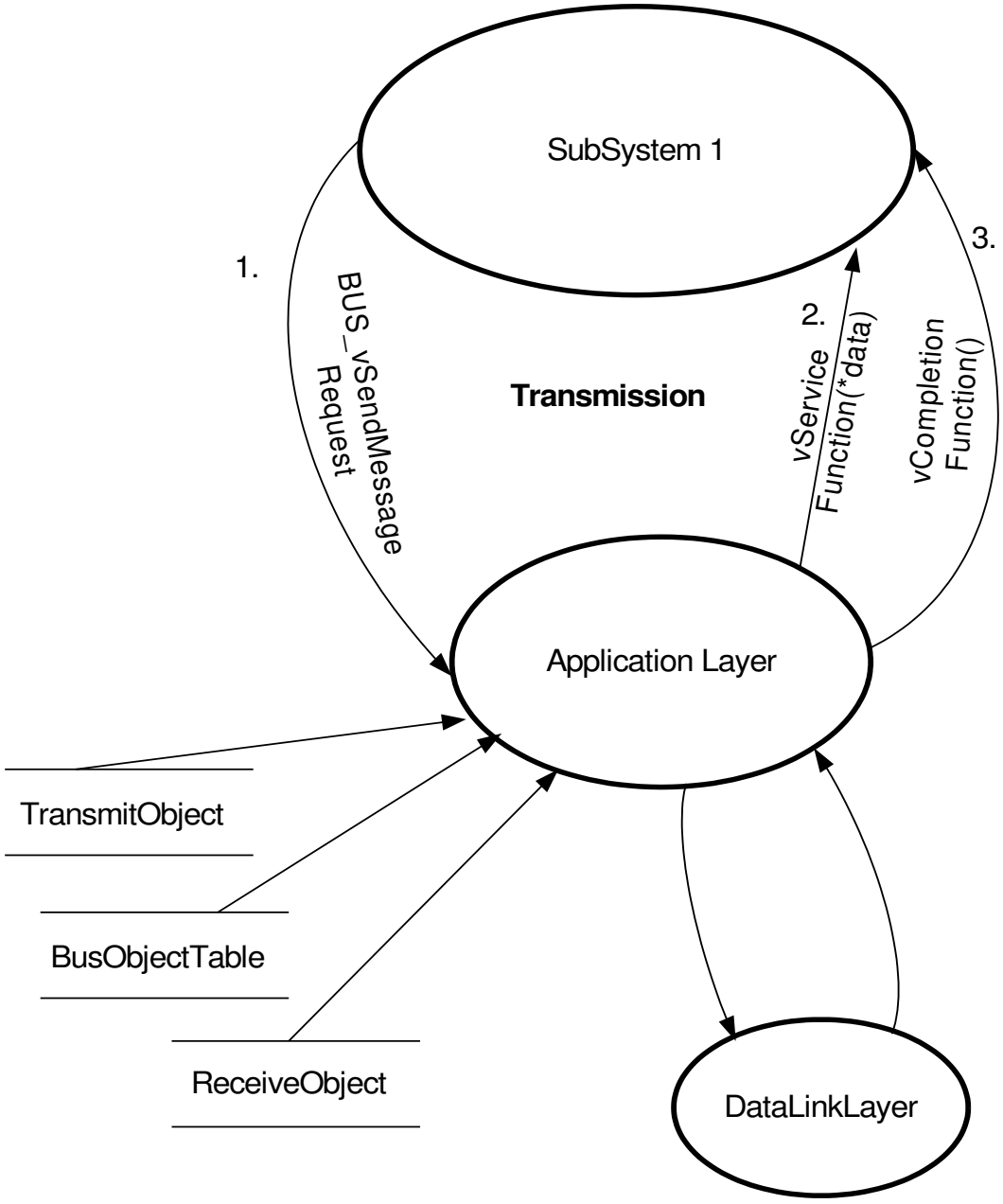


Figure 15: Message transmission

If a completion function is registered (entry in the message transmission table unequal to zero), this is called (step 3. in Figure 15) subsequently.

A possible message prioritisation must be handled by the calling application. Otherwise the messages defined in the first subsystems will typically be handled first (default message prioritisation).

6.1.2 Application layer reception

When a message is received by the bus message handler this is decoded (step 1. in Figure 16) and sent to the associated service function (step 2. in Figure 16) via the ReceiveObjectSubsystem corresponding to the target subsystem. The target subsystem is known from the TargetAddress contained in the frame. Appending the message identifier to the TargetAddress, the bus identifier is generated (bus identifier = 8bit(0) + 8bit(TargetAddress) + 16bit(MessageIdentifier)). The bus identifier is checked in the table associated with the addressed subsystem. If the identifier is not found, the message is discarded (no information is returned about this to the sender as a positive acknowledgement has already been

sent). Please note that in the subsystem table the bus identifier is redundant. The message identifier would be sufficient – although, to make the bus being easier exchangeable with e.g. a CAN-bus it is of advantage to keep using the full bus identifier.

If the received identifier is known, the message data is transferred to the application via the service function, which is associated with this identifier. Within the service function the application could be signalled that a new message has arrived. There is no explicit completion function available for message reception (as this information can be made available in the service function).

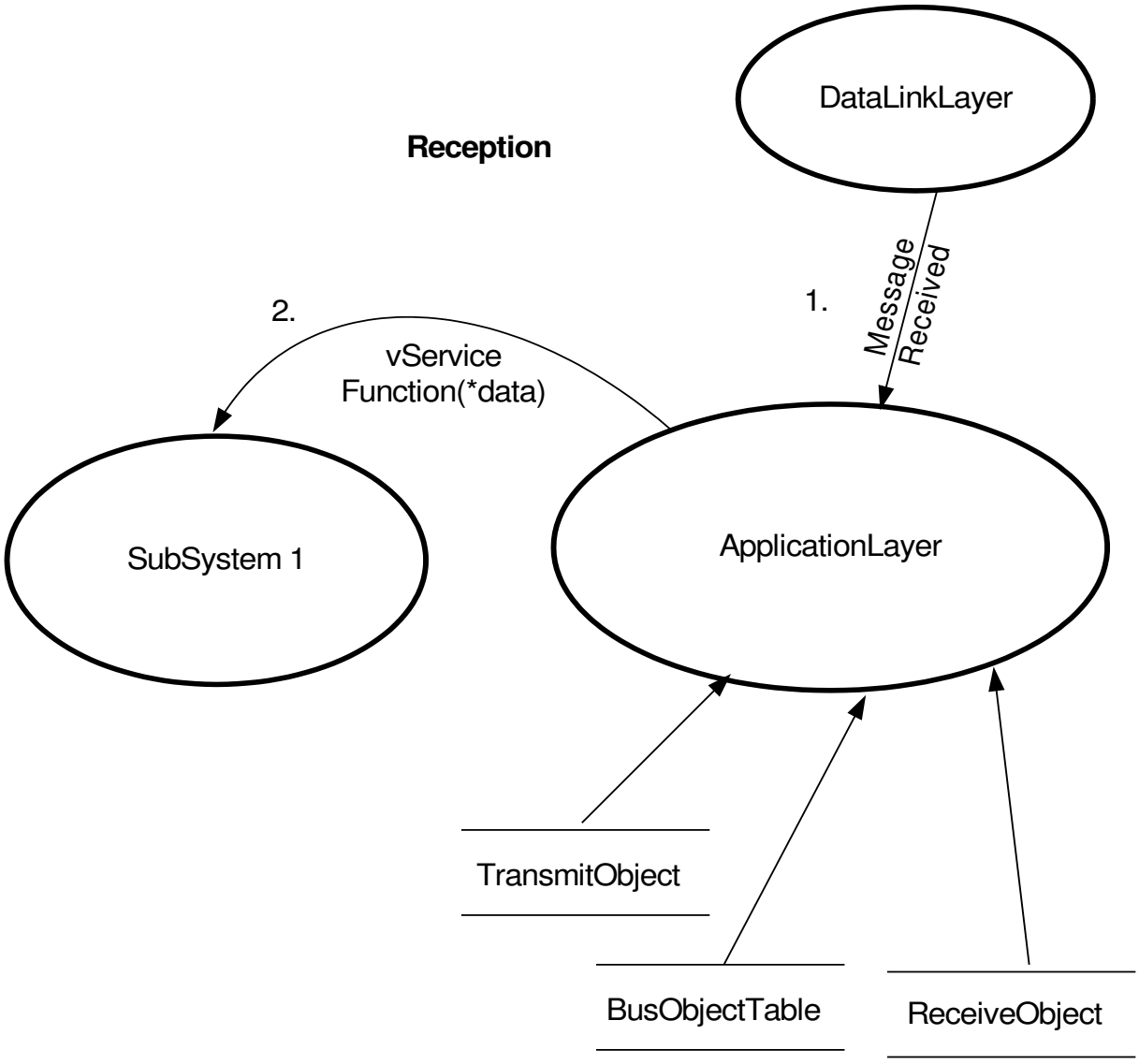


Figure 16: Message reception

6.1.3 Application layer transmittion of Wakeup Signal

Apart from sending and receiving Dbus2 messages, the application layer also has the possibility to send a 15ms (10-20 ms) break signal, in order to wake up nodes, which had previously entered a sleep mode, in which Dbus2 communication has been stopped. The reasons, for which any node might have entered such a sleep mode are completely application specific.

Any node, which has been woken up by a Wakeup signal, may send a Wakeup Sent Request (chapter 5.2.3.18), as a broadcast, in order to find out, if and by which other node it was to be woken up. If no response is received (even after three repetitions), this means, that this node must return to sleep mode.

If the node receiving a Wakeup signal is already awake, it also may issue the Wakeup Sent Request (chapter 5.2.3.18). In this case, the response is a confirmation, that the present state is still correct. If no response is received (even after three repetitions), this means, that this node should not do any action / stay in present mode.

The node, which has sent the Wakeup signal and has received a Wakeup Sent Request (chapter 5.2.3.18), must issue a Wakeup Sent Response (chapter 5.2.3.19), if the node sending the request really is to be woken up or supposed to be already awake.

It is strongly recommended to process the decribed communication handshake for at least one node in the system, to confirm, that the wakeup signal has been correct.

The following sceme is demonstrating the logic described above:

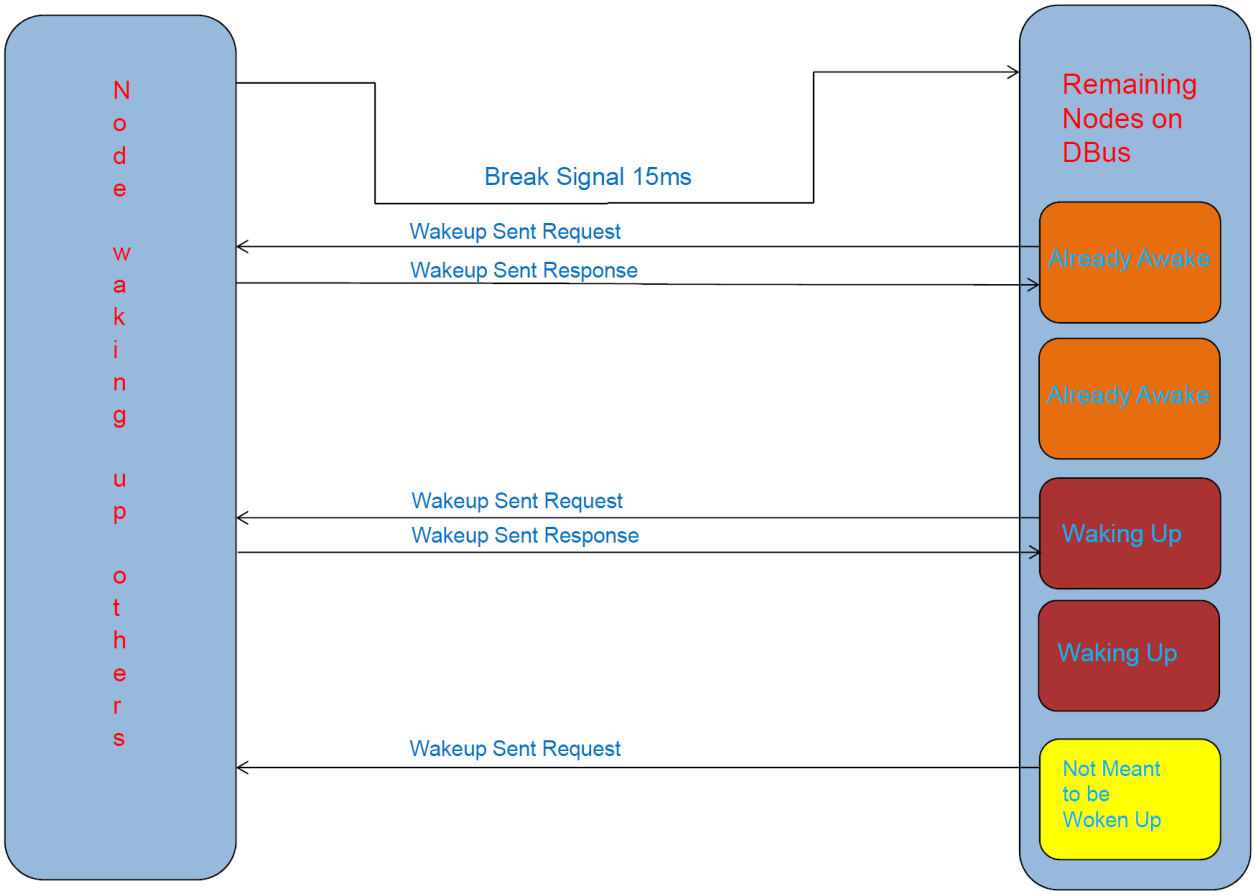


Figure 17: Illustration of using 15ms (10-20 ms) Wakeup break. All nodes receiving the Wakeup may send a Wakeup Sent Request broadcast, in order to find out who has woken them up. The orange nodes are already awake, so the (directed) response is just a confirmation, that the present state is correct. The red nodes get the confirmation (directed response), that they were meant to be woken up. The yellow node does not get any response and returns to sleep mode.

6.1.4 Application layer transmittion of Reset Signal

In order to reset other participants with software problems, the sending of a reset break signal might be triggered by the application layer. The signal consists of two break signals with the lengths of 7 seconds and 2 seconds, respectively. Between the breaks, there is a pause of 750 ms. The receiver has to be connected to a hardware circuit, which will translate the reset breaks into pulling the reset pin of the receiver. This solution is intended to solve potential problems during operation on controllers with full operating systems.

6.2 Application specific degree of freedom

The following application specific issues must be clarified for each system.

- Message identifiers (numeric value)
  The range of different identifiers (relations) must be determined.
  One identifier should only exist once within a system – once for the transmitting node, and once for each possible receiving node.
- Size of the input buffer
  The maximum length of a message, which can be received, depends on the size of the buffer.
- Target addresses
  The addressing of messages is taken care of by the target address, which is defined in the transmit object. The application does not need direct information about how to address other communication partners for being able to exchange system messages.
- Subsystems
  The application can address a system message directly to another subsystem (on another communication partner). This simplifies distribution of the received messages. Default subsystem is 0, which is always used by e.g. service messages. A message sent from a module can be directed to subsystem SENSOR on another communication partner.
- If a message identifier is sent to the wrong subsystem it is unknown, unless one message identifier is defined more than once in the system (this is not allowed according to this specification, but the system is error tolerant towards this issue).
- Baudrate
  A suitable baudrate must be selected. The baudrate must be known in the system so that there is no need to develop baudrate checking or baudrate adjusting processes for the controllers.
- Main hardware node
  The main hardware node supplies the other nodes with V_{cc}. Accordingly, this node's D-Line has a (main) pull-up resistor towards 5 V.
- Number of sending attempts in case of collision, recommended value is 16.
- Number of sending attempts in case of missing ack, recommended value is 4.
- Number of sending attempts in case of negative ack, recommended value is 2.

6.3 Example of Data Exchange

```
See section 2.5 for typedefs

// ***************************************************************
// The following code could be located in file "motor_control.c"
const TbusReceiveObject MOT_tReceiveObject[] = {
  // Bus-ID, Length, ServiceFunction
  { 0x00310120, 1, MOT_vSetMotorErrors       },         //The Id is coded for D-Bus-2: 0x31 means own
                                                           Communication partner is #3 own subsystem is #1 message
                                                           is 0x0120
  { 0xFF310121, 2, MOT_vSetMotorTargetValues },         //The Id is coded for D-Bus-2: 0x31 means own
                                                           Communication partner is #3 own subsystem is #1 message
                                                           is 0x0121, MSB of first byte is end mark.
};

const TbusTransmitObject MOT_tTransmitObject[] = {
  // Bus-ID, Length, ServiceFunction,      ConfirmationFunction
  { 0x00120123, 1, MOT_vGetMotorSpeed,     MOT_vConfirmMotorSpeed }, //The Id is coded for D-Bus-2:0x12 means
                                                                       communication partner #1 subsys #2
                                                                       message #0x0123
                                                                  //for the CAN the meaning would be
                                                                       Id=0x12000123
  { 0x00120124, 2, MOT_vGetMotorDirection, 0            }  //The Id is coded for D-Bus-2:0x12 means
                                                                       Communication partner #1 subsys #2
                                                                       message #0x0124
                                                                  //for the CAN the meaning would be
                                                                       Id=0x12000124
};

#define MOT_MSG_MOTOR_SPEED     0
#define MOT_MSG_MOTOR_DIRECTION 1
```

```
//Example:
BUS_vSendMessage(MOT_SUBSYS,MOT_MSG_MOTOR_SPEED);
//would flag the message MOT_MSG_MOTOR_SPEED for transmission

typedef struct {
  uint16 uiMotorSpeed;
} TmotorSpeed;

void MOT_vGetMotorSpeed(uchar ucDataLen, uchar *pucData) {
  ((TmotorSpeed*)(pucData))->uiMotorSpeed = MOT_uiCurrentMotorSpeed;
}

void MOT_vConfirmMotorSpeed(void) {
  // Message just sent on the bus
}
// *************************************************************


-------------------------------------------------------------------------------


// Receiving would work the following way (using interrupts):
// 1. frame data will be received in interrupt
// 2. frame will be decomposed and passed to main message/service dispatcher
// 3. message will be routed to specific message/service routines through specific routing tables, which may be
//    located in serveral target subsystem modules or in a single module in case of non hieracical designs.
// 4. message will be passed to message/service routine
// SubSys: In case of the D-Bus-2 the SubSys Number will be provided by the hardware interface module. In case of
//       the CAN-Bus a SubSys Number may be generated from the ID-Number e.g. most significant byte and may be
//       provided the same way.


// Sending would work this way:
// 1. sending will be initiated by calling BUS_vSendMessage(TransmitObject* Object) service
//    which causes the bus handling module to remember that this message should be sent, but not to send this
//    message immeadiatly
// 2. the send task periodicaly checks which messages have to be sent and if possible sends them to the hardware
//    interface
```

7 Network Management (NMT) deprecated (new: part of Common Components Messages)

Please note, that functionalities described in this chapter are still supported for software, which is older, than V1.8 of Dbus 2.2 specification. Using some of the Common Components message identifiers, the System States library, which follows more modern concepts, is being developed. It is intended to eventually replace NMT by parts of System States functionality. In the meantime, both concepts are to exist in parallel for several years.

The Network Management is not a multi-master system, but a master-slave system. One node has the role of the master. This node handles the network management functions like connecting or disconnecting nodes – as well as controlling the exchange of data between nodes during the set-up phase of the network (this feature is, however, normally not used). After the network is operational[14], all nodes may initiate a data exchange (i.e. send a message).

The NMT-master keeps an image of its own state in the network object list. When (in this chapter) referred to master, the NMT-master is meant, when referred to slave or node, NMT-slave is meant.

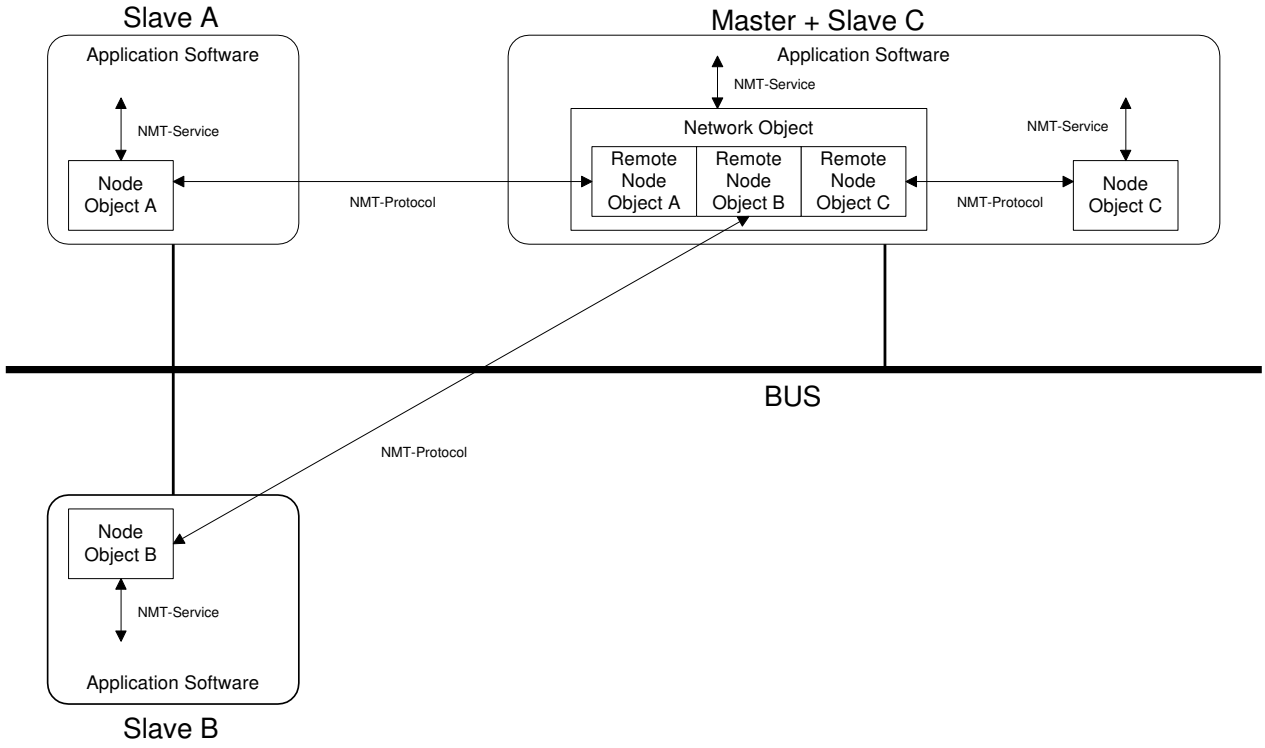


Figure 18: Example of a network showing different attributes of the Network Management

7.1 NMT vs Sleep Mode deprecated (see explanation at start of chapter NMT)

Section 7.5 describes possible levels of network management as described by this specification up to version 2.14. This does not reflect the sleep mode handling. Sleep mode has been defined to work with the described network management (according to section 7.5), but is also possible without any other network management – hence the minimum NMT implementation does not contain anything else than sleep mode and wake-up handling. These messages are defined in section 7.6.2.

[14] see the next paragraphs for an explanation of the term operational.

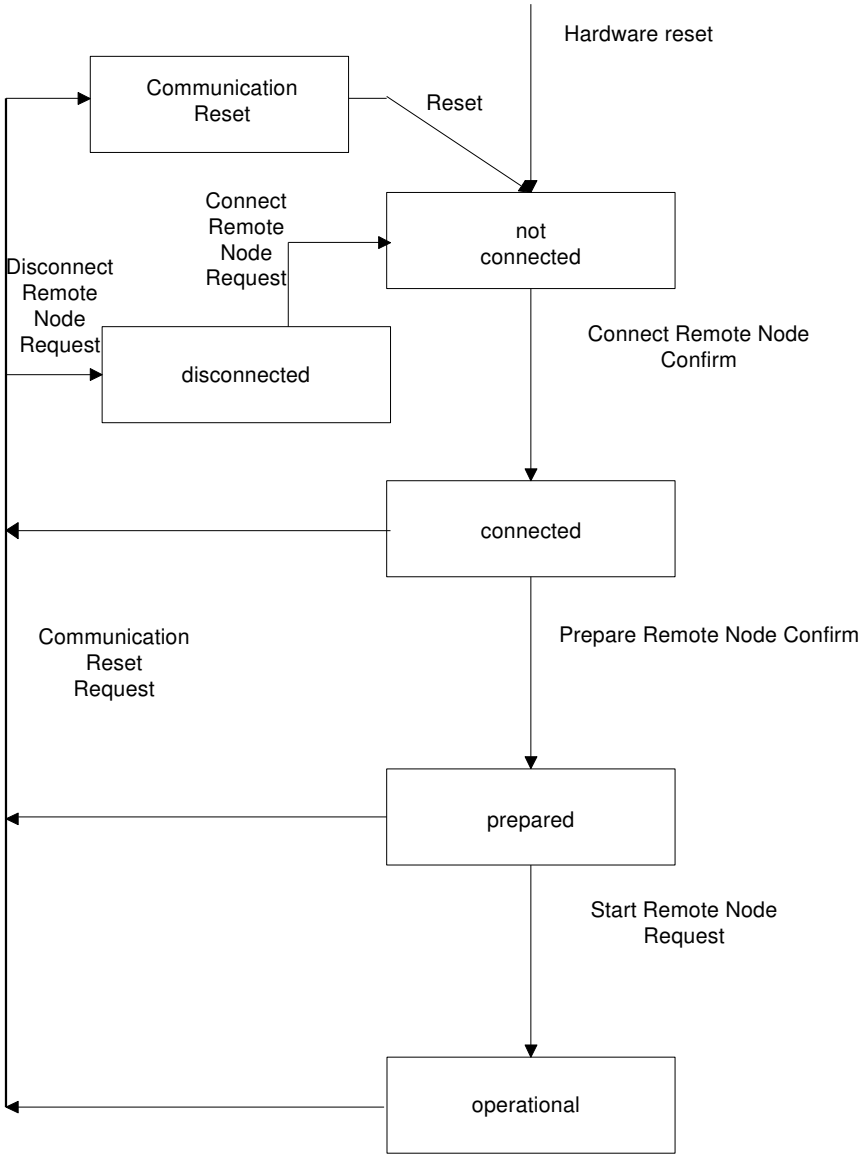
7.2 Attributes of the Network Objects deprecated (see explanation at start of chapter NMT)

- **Network Object:** The Network Object contains several Remote Node Objects.
- **Remote Node Objects:** The Remote Node Object contains information about the state of the remote node, and the reaction to a not available node object, according to the following categories:
    - Essential: All essential nodes enter the operational state simultaneously. The nodes are supervised by the master. Whenever an essential node is not responding properly, a network reset is executed.
    - Temporary essential: These nodes behave as the essential nodes when they are running (set-up). The nodes may, however, be disconnected (logged off). In the disconnected state the master does not care about the node (no supervision, hence no network reset by no response).
    - Non-essential: These nodes are set-up as the previously mentioned nodes, but no network reset is executed if they stop responding.
- **Node Objects:** The node object contains information about its NMT-Address and its present state.

7.3 Network State Transitions (Master view) deprecated (see explanation at start of chapter NMT)

The network is controlled by the NMT master, which has got a state transition diagram for the remote nodes seen in Figure 19. The nodes have got their own state transitions corresponding to the diagram in Figure 20.

Figure 19: Image of the Remote Node



Each node starts in the state NotConnected (essential nodes) or Disconnected (nonessential nodes). The state connected is taken when a SetupNodeConfirmation (with data content: Connect) is received, but only if the remote node is already in the state NotConnected (the Connected state cannot be reached from the state Disconnected, the remote node stays in the Disconnected state).

The State connected may have a substate called preparing. This state is entered when the prepare remote node request is received by the remote node, but the node is still not ready to enter the prepared state (e.g. because the initialisation of the node is still not finished). The master node knows that the remote node is in the state preparing when the remote node upon being requested to enter the prepared state sends node guards instead of prepare confirmation (see next section for a description of node guards). To the master preparing is equivalent to connected. When the node (eventually) answers the prepare remote node request (this request is not repeated) with a prepare node confirmation, the remote node enters the prepared state. Please note that the preparing state is normally not used.

When all essential slaves have entered the prepared state, the master sends a broadcast message, NMT_StartRemoteNodes. As this is a broadcast message, it is not answered (neither acknowledged by any receiver). All nodes enter the operational state. The complete network is considered to be operational.

After reset the network is set-up by first setting up the essential slaves (essential slaves are not allowed to be missing, hence they must always be present). When the essential slaves are all operational, the network itself enters the operational

state. From this point on, all essential nodes are operational, and they are all allowed to send messages on own initiative (before the network is operational only the NMT-master is allowed to initiate information exchange).

The temporary essential and nonessential nodes may now be included in the network. All remote nodes, which are in the state not-connected, enter the connected state when a ConnectNodeConfirm is received. Remote nodes, which are in the state disconnected, are not considered, until explicitly (re)connected (initiated from the application).

If an error occurs (see section 7.4 for a discussion on use of node guards), the network may have to reset. The broadcast message NMT_CommunicationReset (see section 7.6.1.6) is used for notifying all nodes on the network about a reset.

7.3.1 State transitions (Node view) deprecated (see explanation at start of chapter NMT)

After power-on all nodes stay in the state not-connected (or disconnected) until they are requested to enter connected state by the master.

Upon receiving a Communication Reset Request all nodes on the network enter the Communication Reset state. This is a waiting state. The time to wait may be chosen in the range of several seconds. This eases the debugging of the network in the case of network problems[15]. After the timer has run down, a reset follows (depending on the application this can be a hardware reset, software reset or just a network reset).

[15] If the reset is done after waiting e.g. 5 seconds, a human user can easily recognise a network reset, as 5 seconds gives the user enough time to observe other aspects of the system. If the reset should follow without any delay, the system would make resets continuously, making the debugging very hard.

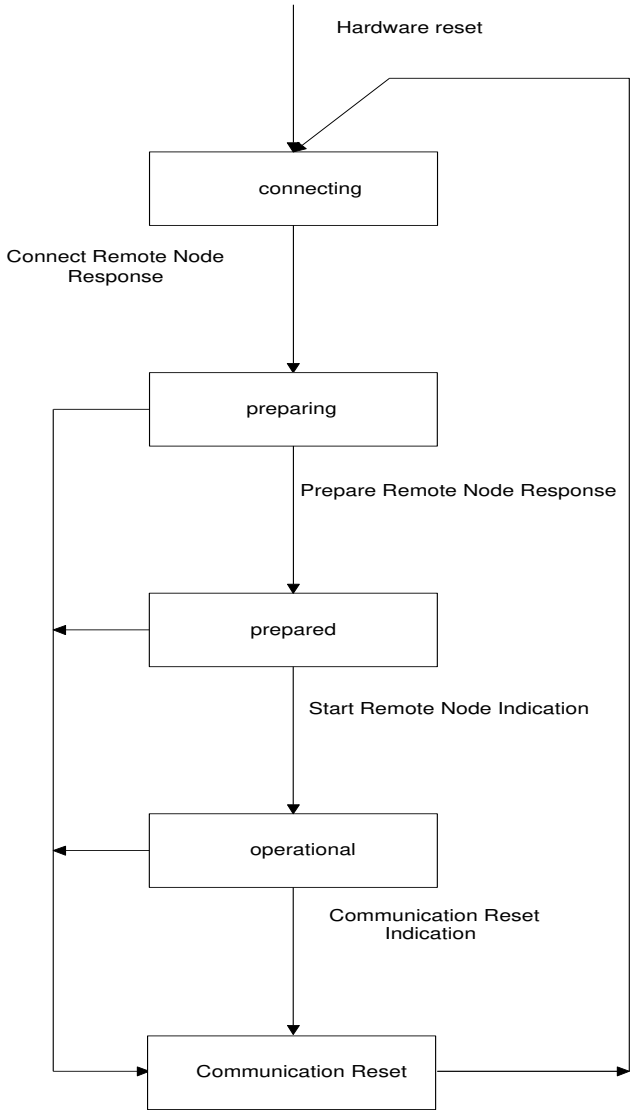


Figure 20: NMT-Slave state transition diagram. Image of the node.

7.4 Node Guarding deprecated (see explanation at start of chapter NMT)

When a network offers priority differences between different messages, i.e. if two messages are scheduled for transmission simultaneously, the higher priority message will be sent, and the lower priority will have to wait till the node sending the higher priority message has stopped the transmission. This is not the case for CSMA/CD (which includes the D-Bus-2), but for e.g. CAN. The network management offers a key feature (from the lower OSI layers) called node guarding. If one node stops responding, this is automatically detected by the NMT-Master, which will either reset the complete system (if the node is essential to the system), or try to reconnect (set-up) the failing node again (if the node is a nonessential to the system). The node guards should typically be received in intervals as fast as the most frequently sent system message on the network.

For networks using prioritisation, the node guards guarantee that no message gets lost, as long as all node guards are exchanged. This lays a foundation for real time systems, guaranteeing that a message will be sent within a certain time, $t_{NodeGuardPeriod}$, describing the time between two node guards.

However, if the bus does not offer any prioritisation of messages (opposed to CAN bus), the node guards cannot guarantee all sent messages to arrive, it can only make the detection more probable (the more frequent the node guards the more probable it is that no message gets lost undetected).

Node guards are exchanged in the following states:

- Connected and Preparing
- Prepared
- Operational

If a slave stops sending node guards it is perceived as not responding any more. If the node is essential, a network reset request will be released from the master (NMT_CommunicationReset message). If the node is nonessential, the master will set-up this node again as soon as it reports (NodeConnectConfirm received).

To be able to recognise that the master is not responding any more, also the master sends node guards. The master sends the NodeGuardMaster message (see section 7.6.1.1). The slaves will evaluate this. If the master stops sending the NodeGuardsMaster message, the slaves will perform a reset – depending on the configuration slaves may inform the other nodes via the CommunicationReset message to execute a reset, as the master would do.

Typical intervals of sending NodeGuards are 100, 200, 500 and 1000 ms. Each node should use the same interval, although it is also possible to differentiate. The system may tolerate a few missing node guards (if the criterion is too strict resets may be released too early…). Assuming a node guard interval (sending) of 100 ms and the monitoring interval of 500 ms the system will tolerate 4 successively missing node guards.

7.5 Level of Network Management deprecated (see explanation at start of chapter NMT)

The Network Management can be restricted to node guarding and set-up of nodes (lower layers), but it can also include high level functions like disconnecting a node, or reconnecting a disconnected node[16].

If the Network Management is added on the level of the application layer, the use of the Network Management stays independent of the underlying protocol and hardware (it does not matter to the application whether the bus is a CAN bus or a D-Bus-2), except for some aspects like the node guards being able to guarantee the arrival of sent messages or not.

Disconnection of a node means that the remote node resets and stays in the state disconnected. The node resets (hardware reset, software reset or network reset – depending on the application on the node), and enters the not-connected state, where it is ready to be set-up by the NMT-master again.

Care should be taken in choosing the node guard interval valid for each system. A system based on fast node guard exchanges will raise the bus load (and correspondingly the processing load). The node guard sending interval is not dependent on the monitoring interval (i.e. node guards may be sent every 500 ms, although a reset is initiated earliest 1000 ms after last received node guard).

When a level of implementation is chosen it is important that all nodes in the network incorporate this. However, it is possible to include a node with a very simple network management (minimum implementation), e.g. a sensor node, when this is considered by the network management master, i.e. the master marks the simple node as nonessential.

Refer to Table for a description of the interface functions.

7.5.1 Minimum Network Management implementation deprecated (see explanation at start of chapter NMT)

The network management may be reduced to only handle resets. This minimum implementation handles a synchronised boot up after reset.

Reset may be requested by using the message ResetExecute (see section 5.2.3.4). Referring to Table, only two interface functions are mandatory.

[16] A not-connected node will automatically be set-up, whereas a disconnected node will not be set-up, until explicitly demanded.

7.5.2 Maximum Network Management implementation deprecated (see explanation at start of chapter NMT)

The "NMT-full-package" contains node guards, including monitoring of all nodes in the network, synchronised set-up after reset and disconnection/reconnection of nodes which are not always needed.

Further, a restriction may be made, to allow for system messages to be exchanged only once within a node guard interval, however, this aspect is not handled by the D-Bus-2. A message may be distributed arbitrarily often (this, of course, demands the application not to continuously initiate a new message distribution).

7.5.3 NMT Pros and Cons deprecated (see explanation at start of chapter NMT)

Advantages of network management:

- Synchronised set-up of all nodes in the network (system messages may only be exchanged when the network has entered the operational state, i.e. when all essential nodes are operational)
- Monitoring of errors on the network (especially profound errors)
- Possibility to disconnect certain nodes still monitoring the complete network

Disadvantages of network management:

- Node guards (and other NMT messages) establish a base load on the bus (without contributing to any information exchange)
- Processing resources are needed
- Implementation code is needed

7.6 NMT Data Structure deprecated (see explanation at start of chapter NMT)

The data structure used for notifying states is shown in the following table:

Numerical Value of State	NMT State	Description
0	NotConnected	No notification between nodes – as no connection is present (the NMT-Master image has this value before a node is set-up).
1	Connected	Used to notify the first state taken by the slave when connecting to the network.
3	Preparing	This state is notified by the slave, when the node is not ready to enter the prepared state immediately after connected (in this state initialisation data may be exchanged between the communication partners). Please note the numeric value of Preparing and Prepared… This state is normally **not used**.
2	Prepared	In this state the node is prepared to enter the operational state. The node will stay in this state until the master sends the StartRemoteNode broadcast.
4	Not used	
5	Operational	This is the normal state, where the application is allowed to send system messages. Whether system messages for initialisation is needed, and whether not allowed messages are ignored must be determined for each system (e.g. a few system messages may be defined for exchange during the initialisation, and all other system messages will be ignored until the network is operational).
6-20	Not used	

Numerical Value of State	NMT State	Description
21	Communication Reset	This is a waiting state. The waiting time can be chosen in the range of several seconds to make the debugging by network problems easier. Please note that the waiting time should be consistent over all the communication partners to avoid repetitive resets.
50	DisconnectNotify	Used as command to enter disconnected state (hence this is no real state)
51	Disconnected	State similar to NotConnected, but explicitly used for avoiding connection of the current node. NMT master may never obtain this state.
127	Offline	State used when NMT is not to be used any more, e.g. after receiving NMT-Shutdown request. This state is useful e.g. when a software update is running.

Table 23: NMT States

7.6.1 NMT Messages deprecated (see explanation at start of chapter NMT)

NMT Messages are mostly (exception: disconnection of a node is initiated from the application layer) transmitted directly from the Data Link Layer.

7.6.1.1 Msg NMT_NodeGuardRequest deprecated (see explanation at start of chapter NMT)

This message is sent from the master as a broadcast to the slaves. The slaves will execute a reset, if the master stops sending these messages. See Table 23: NMT States for a description of parameter 1.

Message Identifier	Msg Parameter 1 NMT-State
E000h MSG_NMT_NODE_GUARD_REQUEST	<1..5>

Length: 3 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier  tMsg;    //Message identifier
  uchar                  ucState; //1..5 (1 is connected, 5 is operational)
} TnmtNodeGuardMaster;
```

7.6.1.2 Msg NMT_NodeGuardConfirm deprecated (see explanation at start of chapter NMT)

One NodeGuardConfirm is sent from each communication partner (broadcast may be used, only the master will evaluate these messages). The second data byte notes the address of the responding slave. This way, it is explicitly known, which nodes are present, and if possibly some nodes are not responding any more. The NMT-Address lies between 1 and 15 (refer to section 4.5). The NMT-Master checks whether the transmitted state (Message parameter 1) is consistent with its image of each node.

Message Identifier	Msg Parameter 1 NMT-State	Msg Parameter 2 NMT-Address
E001h MSG_NMT_NODE_GUARD_CONFIRM	<1..5>	<1..15>

Length: 4 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier  tMsg;    //Message identifier
  uchar                  ucState; //1..5, 0x33 (1 is connected, 5 is operational, 0x33 is disconnected)
  uchar                  ucSlaveAddress;  //1..F
} TnmtNodeGuardConfirm;
```

7.6.1.3 Msg NMT_SetupRemoteNodeRequest deprecated (see explanation at start of chapter NMT)

The message is sent as broadcast, however the addressed node (the node which will be set-up) is stated in the second data byte. Possible values of parameter1 are 1 (Connect Remote Node) and 2 (Prepare Remote Node). See also Table 23: NMT States.

Message Identifier	Msg Parameter1	Msg Parameter 2
	StateTransition	NMT-Address
E700h MSG_NMT_SETUP_REMOTE_NODE_REQUEST	<1..2>	<1..15>

Length: 4 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier  tMsg;                 //Message identifier
  uchar                  ucStateTransition;    //1: ConnectRemoteNode, 2: PrepareRemoteNode
  uchar                  ucSlaveAddress;       //1..15
} TnmtSetupRemoteNodeRequest;
```

7.6.1.4 Msg NMT_SetupNodeConfirm deprecated (see explanation at start of chapter NMT)

The message parameter 2 contains the address of the node, which sent this message (as the nodes do not need to know the address of the master, they may transmit this message as a broadcast). See Table 23: NMT States for a description of NMT-States.

Message Identifier	Msg Parameter1	Msg Parameter 2
	State	NMT-Address
E800h MSG_NMT_SETUP_REMOTE_NODE_CONFIRM	<1..5>	<1..15>

Length: 4 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier  tMsg;           //Message identifier
  uchar                  ucState;        //1: Connected, 3: Preparing, 2: Prepared
  uchar                  ucSlaveAddress; //Address of the node (sender of this message)
} TnmtStartRemoteNode;
```

7.6.1.5 Msg NMT_StartRemoteNodes deprecated (see explanation at start of chapter NMT)

Broadcast with the request to all nodes in the prepared state to enter the operational state.

Message Identifier
EFFFh MSG_NMT_START_REMOTE_NODE

Length: 2 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;     //Message identifier
} TnmtStartRemoteNode;
```

7.6.1.6 Msg NMT_CommunicationReset deprecated (see explanation at start of chapter NMT)

Broadcast to all nodes to execute a reset. A second data byte is optional – this can be used for NMT master as an indication of which node stopped responding.

Message Identifier	Msg Parameter 1 NMT-Address	Msg Parameter 2 Not Responding Node
EFFEh MSG_NMT_COMMUNICATION_RESET	<1..F>	<1..15>

Length: 3/4 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier   tMsg;                 //Message identifier
  uchar                   ucNodeAddress;        //NMT-Address of the own node.
  <uchar                  ucNotRespondingNode;  //Optional byte>
} TnmtStartRemoteNode;
```

7.6.1.7 Msg NMT_ShutDown deprecated (see explanation at start of chapter NMT)

This message is used to force the network management to enter an idle state. In the case of updating a communication partner, it is important that this (or any other in the network) does not suddenly execute a reset due to missing node guards (refer to the description of message 5.2.3.2). Also for debug reasons it might be convenient to stop the network management, e.g. before running into a breakpoint or disconnecting the bus.

A reset is needed to start the network management again (e.g. by using the message ResetExecute described in section 5.2.3.4).

Message Identifier
EEEEh MSG_NMT_SHUT_DOWN

Length: 2 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;     //Message identifier
} TnmtShutDown;
```

7.6.2 NMT Messages – sleep mode deprecated (see explanation at start of chapter NMT)

The sleep mode messages can be used in combination with the other network management messages – as described by this chapter – but also as stand-alone version (i.e. no other NMT-funtionality than sleep mode handling).

7.6.2.1 Msg NMT_EnterSleepMode deprecated (see explanation at start of chapter NMT)

This message is used to request nodes to enter sleep mode. Nodes, which receive this message, will enter a state, where it is no longer possible to understand normal D-Bus-2 messages. For waking up, a special signal is expected, which is described by reference [4]. This way, it is possible that some nodes continue normal operation (communicating via D-Bus-2), which other nodes are logically absent (in sleep mode).

Message Identifier
EA00h MSG_NMT_ENTER_SLEEP_MODE

Length: 2 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;     //Message identifier
} TnmtEnterSleepMode;
```

7.6.2.2 Msg NMT_WakeUp deprecated (see explanation at start of chapter NMT)

When one or several sleeping nodes should enter normal operation, a wake-up signal is expected. This signal is described by reference [4], which also describes circuitry for generation and recognition of this signal. This message should follow this signal after a, project specific, short delay allowing nodes to physically enter an operating state. Nodes, which receive this D-Bus-2 message, are confirmed that they are meant to wake-up (whereas the wake-up signal will be received by all

sleeping nodes, also nodes, which potentially should stay in sleep mode). Nodes, which do not receive this message as confirmation to wake-up, are expected to return to sleep mode after a certain time (project specific).

Message Identifier
EAAAh MSG_NMT_WAKE_UP

Length: 2 Byte

Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;     //Message identifier
} TnmtWakeUp;
```

7.7 NMT Interface deprecated (see explanation at start of chapter NMT)

The application can use the network management to set-up remote nodes (for the NMT master) or check if the network is operational. The network management must, however, also be able to use specific functions provided by the application, e.g. reset.

The interface described in this section is the interface towards the application. The messages described in the previous section (7.6) form the interface towards the other nodes (via the bus).

Function	Parameter 1	Mandatory	Master/Slave	Direction[17]
Reset	uchar NodeAddress	X	Master & Slave	NMT->user
Is Node Operational		X	Slave	user->NMT
Configure Network Objects			Master	user->NMT
Disconnet Remote Node	uchar NodeAddress		Master	user->NMT
Is Remote Node Disconnected	uchar NodeAddress		Master	user->NMT
Connect Disconnected Node	uchar NodeAddress		Master	user->NMT
Is Remote Node Operational	uchar NodeAddress		Master	user->NMT

Table 24: NMT Service Interface

7.7.1 NMT Slave interface deprecated (see explanation at start of chapter NMT)

In the following section the slave interface is described. This interface is mandatory for master and slave. C-code is illustrative.

7.7.1.1 NMT Reset deprecated (see explanation at start of chapter NMT)

The application can use the network management to set-up remote nodes (for the NMT master) or check if the network is operational. The network management must also be able to use specific functions provided by the application, e.g. reset.

Through the following function:

```
NMT_vResetExecute(uchar ucNodeAddress)
```

The network management will execute a reset. This can be done by registering the reset function in main.c or by overwriting the function provided in the NMT module for direct access to the corresponding function in main.c (or wherever the reset function is implemented).

[17] The term user refers to the overlying layer which is using this layer.

7.7.1.2 NMT IsNodeOperational deprecated (see explanation at start of chapter NMT)

Whether this node is ready to exchange normal process data can be checked by calling the following function:

```
NMT_ucIsNodeOperational(void)
```

7.7.2 NMT Master interface deprecated (see explanation at start of chapter NMT)

The functions, which the application may need, regarding the state of the nodes in the network, are mostly present only on the NMT master node. C-code is illustrative.

7.7.2.1 NMT ConfigureNetworkObjects deprecated (see explanation at start of chapter NMT)

The NMT master supplies an initialisation function, where the NetworkObjects are configured:

```
NMT_vConfigureNetworkObjects(void)
```

This configuration contains information for each node in the network about the following:

- Initial state (either NotConnected or Disconnected)
- Importance of the node (node type – e.g. essential). See description in section 7.2.

7.7.2.2 NMT Disconnect Node deprecated (see explanation at start of chapter NMT)

The application can disconnect a node from the network by calling the following function:

```
NMT_vDisconnectNode(uchar ucNodeAddress)
```

Correspondingly, the following function tells whether a node has been successfully disconnected or not:

```
NMT_ucIsNodeDisconnected(uchar ucNodeAddress)
```

7.7.2.3 NMT ConnectDisconnectedNode deprecated (see explanation at start of chapter NMT)

A disconnected node may be connected again by calling the following function:

```
NMT_vConnectDisconnectedNode(uchar ucNodeAddress)
```

Please note that a node, which is in the state Disconnected, will not be set-up by the master. A node, which has been disconnected, will typically perform a reset, and cyclically send a ConnectConfirm (“I’m there”) to the master. The master, however, will not react to this message, unless the node has transitioned from the Disconnected state to the NotConnected state (initiated by this function).

A node which per default is in Disconnected (configuration via Network Object start-up state) state will not be set-up until the ConnectDisconnectedNode function has been called.

7.7.2.4 NMT IsRemoteNodeOperational deprecated (see explanation at start of chapter NMT)

Whether a remote node is ready to exchange normal process data can be checked by calling the following function:

```
NMT_ucIsRemoteNodeOperational(uchar ucNodeAddress)
```

8 Statistical information (Optional)

Statistical information must always be available for debugging purposes (as well as for performance tests) during the development phase. In the software released for the series/final release the statistics are optional.

Statistical information must be available for the layers, which are implemented, i.e. if NMT is implemented, statistics regarding the NMT must be available. NMT is deprecated (see explanation at start of chapter NMT).

8.1 Statistical information messages

Statistical information can be collected via messages defined in the following table:

Software Layer	Message • Content	Counter	Type	Variable content kept over Reset
DLL	Data link statistics (errors) • Collisions • CRC • Ack-timeout • Nack • Interbyte-timeout	 X X X X X	 Byte Byte Byte Byte Byte	
NMT	NMT Slave errors • Last Event • Reset Execute Received • Nonresp. master	 X X	 Byte Byte Byte	 X X X
	NMT Master errors • Last Event • Reset Execute Received • Nonresp. remote node	 X X	 Byte Byte 1 Byte per slave	 X X X
	NMT Master Network Object • NMT_NetworkObject[NUMBER_OF_NODES]		1 Byte per slave	

Table 25: Statistical information messages

For an explanation of last event codes, see Table 26.

8.1.1 Data Link Layer statistics

8.1.1.1 Msg DLL_StatisticsRequest

The message is directed from the pc to the device. Statistical information is returned via the message StatisticsDLL_Response.

Message Identifier
FA00h MSG_DLL_STATISTICS_REQUEST

Length: 2 Byte

Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;     //Message identifier
} TdllStatisticsRequest;
```

8.1.1.2 Msg DLL_StatisticsResponse

The message is directed from the device to the pc. Returns statistical information containing: collisions, CRC, ACK-timeouts, NACK, InterbyteTimeouts, and too long messages. Each statistic is a one-byte counter.

Message Identifier	Param1 Collisions	Param2 CRC	Param3 ACK timeout	Param4 ACK-error	Param5 Interbyte timeout
FA01h MSG_DLL_STATISTICS_RESPONSE	# errors	# errors	# errors	# errors	# errors

Length: 8 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;     //Message identifier
  uchar ucCollisionsCtr;          //Collisions counter
  uchar ucCrcCtr;                 //Error in CRC counter
  uchar ucAckTimeoutCtr;          //Acknowledgement timeout counter
  uchar ucAckErrorCtr;            //Acknowledgement error counter
  uchar ucInterbyteTimeoutCtr;    //Timeout during reception counter
} TdllStatisticsResponse;
```

8.1.2 NMT Statistics deprecated (see explanation at start of chapter NMT)

Several errors may appear which are related to the network management. If one node stops responding, the network (assuming it is an essential node) will perform a reset. To be able to monitor the reason of the reset, it is important to be able to gather this information from the NMT master, as well as from the NMT slaves (e.g. if the master stopped responding).

Most of the statistical information is gathered in counters, which are incrementing each time the corresponding error appears. As it is not easy to figure out exactly what triggered the last reset on the network by analysing the counters, a last event is also offered. The last event offers information according to the following table:

Error Code (in hex)	Last event
00h	NMT Master stopped responding (node guards missing)
01h	Node 1 stopped responding
02h	Node 2 stopped responding
..	
0Fh	Node F stopped responding
10h	NMT Master requests a reset (Communication Reset received)
11h	Reserved
..	
FEh	
FFh	No last event (nothing happened, yet)

Table 26: Last Event error codes

8.1.2.1 Msg NMT_StatisticsRequest deprecated (see explanation at start of chapter NMT)

The message is directed from the pc to the NMT master or slave (depending on the target address used).

Message Identifier
FA02h MSG_NMT_STATISTICS_REQUEST

Length: 2 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;     //Message identifier
} TnmtStatisticsRequest;
```

8.1.2.2 Msg NMT_SlaveStatisticsResponse deprecated (see explanation at start of chapter NMT)

The message is directed from a slave to the pc. Returns statistical information containing: ResetExecute received, nonresponding master and last event. Each statistic is a one-byte counter, except for the last event, which is described in Table 26: Last Event error codes.

Message Identifier	Param1 Last Event	Param2 ResetExecuteReceived	Param3 Nonresponding Master
FA03h MSG_NMT_SLAVE_STATISTICS_RESPONSE	<0..FFh>	# errors	# errors

Length: 5 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;     //Message identifier
  uchar ucLastEvent;              //Last event (reset reason)
  uchar ucResetExecuteReceivedCtr;//ResetExecute received counter
  uchar ucNonrespondingMasterCtr; //Nonresponding master counter
} TnmtSlaveStatisticsResponse;
```

8.1.2.3 Msg NMT_MasterStatisticsResponse deprecated (see explanation at start of chapter NMT)

The message is directed from the master to the pc. Returns statistical information containing: Last event, ResetExecute received, nonresponding master and last event. Each statistic is a one-byte counter, except for the last event, which is described in Table 26: Last Event error codes.

Message 8.1.2.3.1 Identifier	Param1 Last Event	Param2 ResetExecuteReceived	Param3… Nonresponding Slave 1..n
FA04h MSG_NMT_MASTER_STATISTICS_RESPONSE	<0..FFh>	# errors	# errors

Length: 5 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;                          //Message identifier
  uchar ucLastEvent;                                   //Last event (reset reason)
  uchar ucResetExecuteReceivedCtr;                     //ResetExecute received counter
  uchar aucNonrespondingNodeCtr[MAX_NUMBER_OF_NODES];  //Nonresponding slave counter
} TnmtSlaveStatisticsResponse;
```

8.1.2.4 Msg NMT_NetworkObjectStateRequest deprecated (see explanation at start of chapter NMT)

The message is directed from the pc to the NMT master to get information about the network state.

Message Identifier
FA05h MSG_NMT_NETWORK_OBJECT_STATE_REQUEST

Length: 2 Byte
Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;     //Message identifier
} TnmtNetworkObjectStateRequest;
```

8.1.2.5 Msg NMT_NetworkObjectStateResponse deprecated (see explanation at start of chapter NMT)

The message is directed from the master to the pc. Returns the present network state, according to Table 23: NMT States. Please note that one of the NetworkObjects is the master's own slave (network node address x (NMT-Address) = CommunicationPartner x).

Message Identifier	Parameter 1 NetworkObject1..NetworkObjectF
FA06h MSG_NMT_NETWORK_OBJECT_STATE_RESPONSE	State of the remote nodes

Length: 3-18 Byte

Type:

```
typedef struct {
  TbusMessageIdentifier tMsg;                              //Message identifier
  uchar aucNetworkObject[MAX_NUMBER_OF_NODES];     //includes the slave of the master node…
} TnmtNetworkObjectStateResponse;
```

9 Error handling

The following set of rules are to be applied, whenever:

- An unexpected stream of bytes is read from the bus: CRC Error will be stated (the second byte in the stream constitutes an address, if the corresponding node is present, then this node will send an ack-wrong. If a node is not addressed, then the stream will be read (and discarded) until either an acknowledgement time-out appears (fewer data bytes than stated by the message length), or a new message is interpreted (more data bytes than stated by the message length).
- A message is received immediately after another message was on the bus (no idle-time between the messages): The message is treated as if there were a regular idle time between the messages. This way it is easier to resynchronise after a collision (or other reasons for being unsynchronised). Also, it is possible to (exceptionally, of course) allow a node (typically the diagnostic station) to send data in a stream without the mandatory idle time between each message.
- Erroneous CRC is notified (Ack-Wrong): The current message must be regarded as not successfully transmitted. One or more repetitions are expected; service messages are not expected to be repeated, as these are master-slave driven (i.e. repetition on level 7: Application). Amount of repetitions (data link layer level repetitions) of other messages according to project specific needs.
- The addressed communication partner does not answer (missing acknowledgement): The communication partner may be offline or physically absent. Only a few repetitions are expected (many would load the bus).
- Message received with unknown message identifier: Message is discarded (but positivtely acknowledged).
- Messages received with a MessageLength of 0 shall be identified as a D-Bus-3 frame. Such frames cannot be processed by the D-Bus-2.1 or D-Bus-2.2 node and will be ignored, until an idle time (frame if finished) is detected. The D-Bus-2.1 or D-Bus-2.2 node monitors again the line and looks for a new frame.
- A collision appears on the bus: At least one of the transmitting parts perceives the collision, and stops the transmission. If another transmitting node does not perceive any collision, his frame is still valid, hence the transmission continues from this node. Care must be taken to assure that two nodes will not transmit an identical message, as this may not be perceived as a collision by all transmitting nodes. After a perceived collision the transmission is cancelled, and can only be restarted after the mandatory idle time (which increases after collisions).
- Set Memory Module x, where the memory module x is not used by the communication partner: The memory module is mapped to 0, which is always present. An ID-Request will reveal that memory module x is not available, because instead memory module 0 is reported.
- Address out-of-range
    - Erase Block Request: Address is restricted to the valid range, hence the out-of-range part is cut (no erase done out-of-range).
    - Read Access: To be defined independently by each project (i.e. no forced standard behaviour)
    - Write Access: Address is restricted to the valid range, hence the out-of-range part is cut (no writing done out-of-range).
- Write request to ROM address: Writing may be tried (by the access function), any read to check, whether the memory was successfully written will show that the writing procedure is successful only if the written data corresponds to the already present data.